MACHINE LEARNING

Damiano De Sanctis

Dispensa con concetti teorici di ML

Indice

1 Statistical learning theory **4**
1.1 Definizioni e terminologia 4
1.2 Expected risk e funzione di costo 4
1.3 Bayes optimal estimator 6
1.4 Scelta della classe di funzioni 7
1.5 No free lunch theorem 7
1.6 Modelli generativi e discriminativi 7
1.7 PAC Learning 8
1.7.1 PAC Learnability 8
1.8 Consistent learner 9
1.9 Inconsistent learner 10
1.9.1 Agnostic PAC learning 11
1.10 Infinite class 11
1.10.1 Infinite class e shattering 11
1.10.2 VC-Dimension 11

2 Principal Component Analysis **12**
2.1 Richiami di algebra lineare 12
2.1.1 Matrici come mappe lineari 12
2.1.2 Autovalori e autovettori 14
2.1.3 Eigendecomposition 15
2.1.4 Teorema Spettrale 15
2.2 PCA 16
2.2.1 Matrice di covarianza 17
2.2.2 Rotazione e decorrelazione tramite PCA 19
2.2.3 Whitening (sferizzazione) dei dati 19

3 Gaussian Mixture Models **20**
3.1 Distribuzione normale e mixture of gaussians . . . 20
3.2 Probabilità congiunta, marginale e condizionata . . . 21
3.3 Learning dei parametri . . . 22
3.4 MLE per una gaussiana singola . . . 22
3.5 Responsibilities nei GMM . . . 25
3.5.1 Calcolo delle responsibilities . . . 25
3.5.2 Algoritmo Expectation-Maximization per GMM . . . 26

4 Regressione lineare e polinomiale **28**
4.1 Regressione lineare . . . 28
4.1.1 Metodo analitico . . . 29
4.1.2 Metodo probabilistico . . . 31
4.1.3 Metodo algebrico (proiezione) . . . 32
4.1.4 Metodo numerico (gradient descent) . . . 33
4.2 Regressione polinomiale . . . 34
4.2.1 Modello lineare . . . 35
4.2.2 Estensione tramite basis functions . . . 35
4.2.3 Regressione polinomiale . . . 35
4.2.4 Overfitting . . . 36
4.2.5 Regolarizzazione L2 . . . 36
4.2.6 Soluzione analitica e interpretazione spettrale . . . 36

5 Classificazione **37**
5.1 Perceptron e logistic regression . . . 37
5.1.1 Algoritmo del perceptron . . . 38
5.1.2 Logistic regression . . . 40
5.1.3 Apprendimento dei parametri. . . . 42
5.2 Multiclass classification . . . 42
5.2.1 Divergenza tra distribuzioni . . . 43
5.2.2 Score functions . . . 43
5.2.3 Passaggio dal caso binario a multiclasse . . . 44
5.2.4 Ottimizzazzione e softmax . . . 44

6 Reti neurali **45**
6.1 Mini-batch training . . . 45
6.1.1 SGD su mini-batch . . . 46
6.2 Momentum . . . 47
6.2.1 SGD con momentum . . . 47
6.3 Ottimizzazione convessa e non convessa . . . 47
6.3.1 Loss surface in linear regression . . . 47
6.3.2 Deep Learning e ottimizzazione non convessa . . . 48
6.4 Struttura di una rete neurale . . . 48

6.4.1 Rappresentazione di un singolo layer . . . 48
6.4.2 Rete neurale a due layer . . . 49
6.5 Funzione di attivazione ReLU . . . 49
6.5.1 Confronto tra ReLU e funzione logistica . . . 49
6.6 Inizializzazione di una rete neurale . . . 50
6.6.1 Inizializzazione nulla . . . 51
6.6.2 Inizializzazione costante non nulla . . . 52
6.6.3 Inizializzazione casuale asimmetrica . . . 52
6.6.4 Xavier/He initialization . . . 52
6.7 Reti neurali profonde . . . 53
6.7.1 Teorema di approssimazione universale . . . 53
6.8 Backpropagation . . . 53
6.8.1 Differenze finite . . . 54
6.8.2 Descrizione di backpropagation . . . 54
6.8.3 Autodiff e autograd . . . 54

7 Clustering **55**
7.1 Definizione del problema . . . 55
7.2 Algoritmo k-means . . . 57

8 Valutazione di un modello **60**
8.1 Bias-Variance trade-off . . . 60
8.1.1 Bias . . . 60
8.1.2 Varianza . . . 61
8.1.3 Decomposizione bias-varianza . . . 61
8.2 Training set, validation set e test set . . . 61
8.3 Cross-validation . . . 62
8.3.1 LOO cross-validation . . . 63
8.4 Regolare gli iperparametri . . . 63
8.5 Metriche di valutazione per la classificazione . . . 64
8.5.1 Classificazione binaria e multiclasse . . . 64
8.5.2 Accuracy . . . 64
8.5.3 Top-K Accuracy . . . 65
8.5.4 Confusion matrix . . . 65
8.5.5 Precision e recall . . . 65
8.5.6 F-measure . . . 66
8.6 True/False positive rate . . . 66
8.6.1 ROC Curve . . . 66
8.7 Balanced accuracy . . . 66

1 Statistical learning theory

1.1 Definizioni e terminologia

Introduciamo la terminologia necessaria per affrontare la teoria dell'apprendimento statistico.

- **Esempi**: istanze di dati usati per apprendimento o valutazione.
- **Features**: insieme di attributi, spesso rappresentati come vettori, associati ad un esempio.
- **Etichette** (labels): valori o categorie associati agli esempi.
- **Training sample**: esempi utilizzati per addestrare un algoritmo di learning.
- **Validation sample**: esempi utilizzati per ottimizzare i parametri di un algoritmo di learning.
- **Test sample**: esempi utilizzati per valutare la performance di un algoritmo di learning. Il test sample è separato dai dati di validation e training, infatti non sono disponibili nella fase di training.
- **Loss function**: una funzione che misura la differenza tra l'etichetta prevista e quella reale. Supponiamo di chiamare Y il set di tutte le etichette e Y' il set delle possibili previsioni, definiamo la funzione di loss come una mappa $L : Y \times Y' \to \mathbb{R}^+$. Tra i principali esempi di loss function abbiamo *0-1 loss* definita in $\{-1, 1\}^2$ come $L(y, y') = 1_{y \neq y'}$ e *squared loss* definita su $I \times I$ da $L(y, y') = (y' - y)^2$ dove I è solitamente un intervallo limitato.
- **Set di ipotesi** $\mathcal{H}$: insieme di funzioni che mappano i vettori di features al set di etichette.
- **Supervised learning**: il learner riceve un set di esempi etichettati come dati di training e fa delle previsioni sui dati non esaminati. Viene spesso utilizzato per problemi di classificazione, regressione e ranking.
- **Unsupervised learning**: il learner riceve un set di esempi non etichettati e fa delle previsioni sui dati non esaminati. Viene utilizzato spesso per problemi di clustering e dimensionality-reduction.

1.2 Expected risk e funzione di costo

L'obiettivo teorico del machine learning è trovare la funzione f che **minimizza l'errore medio** su tutti i possibili dati generati dalla distribuzione reale. Questo errore medio si

chiama **expected risk** ed è definito come

$$\mathbb{E}\left[L(f(x), y)\right] = \iint_{X \times Y} p(x, y)\, L(f(x), y)\, dx\, dy$$

Questo rappresenta l'errore medio che il modello farebbe su tutti i dati possibili. Il problema è che **non possiamo calcolarlo**, perché nella pratica **non conosciamo la distribuzione** $p(x, y)$ ma soltanto il dataset. Proprio per questo motivo introduciamo l'**empirical risk** cioè una stima dell'errore fatta sui dati del dataset

$$\hat{\mathcal{E}}(f) = \frac{1}{n} \sum_{i=1}^{n} L\left(f(x_i), y_i\right)$$

ovvero la media della loss sui dati di training. Invece l'idea dell'**empirical risk minimization** è quella di scegliere f che minimizza l'errore empirico. Dato il dataset $\{x_i, y_i\}_{i=1}^{n}$ il processo di apprendimento produce una funzione stimata $\hat{f}_n$ che dipende dai dati osservati. Per questo motivo $\hat{f}_n$ è chiamata **estimator**. Per capire quanto il nostro stimatore è vicino alla soluzione ideale confrontiamo il suo errore con il miglior errore possibile. Consideriamo la quantità

$$\mathcal{E}(\hat{f}_n) - \min_{f \in \mathcal{F}} \hat{\mathcal{E}}(f)$$

Questa quantità misura quanto il nostro modello è peggiore rispetto alla migliore funzione possibile. Questa differenza si chiama **excess risk**. Bisogna notare che anche questa è una variabile aleatoria perchè dipende dai dati del training set. Questa quantità misura **quanto il nostro modello è peggiore rispetto al modello ottimale** che potremmo ottenere se conoscessimo perfettamente la distribuzione dei dati. L'excess risk può essere scomposto in due termini:

$$\underbrace{\mathcal{E}(\hat{f}_n) - \hat{\mathcal{E}}(\hat{f}_n)}_{\text{generalization gap}} + \underbrace{\hat{\mathcal{E}}(\hat{f}_n) - \min_{f \in \mathcal{F}} \mathcal{E}(f)}_{\text{lower bound}}$$

Questa decomposizione separa due tipi diversi di errore. Il primo termine si chiama **generalization gap** e rappresenta la differenza tra: l'errore vero del modello (expected risk) e l'errore calcolato sul dataset (empirical risk). Questo termine è importante perché il modello viene scelto minimizzando l'errore empirico, ma quello che realmente ci interessa è l'errore vero. Il generalization gap può essere studiato o controllato in diversi modi. Uno dei metodi è la **cross-validation**. In questo caso il dataset viene diviso in due parti: una parte viene usata per il training (per fare ERM), mentre un'altra parte viene tenuta da parte e utilizzata solo per valutare il modello. In questo modo possiamo stimare meglio quanto il modello generalizza. Un altro approccio è **teorico**: si studia direttamente la differenza tra expected risk ed empirical risk. In particolare si cerca di ottenere un limite superiore dell'errore vero:

$$\mathcal{E}(\hat{f}_n) \leq \underbrace{\hat{\mathcal{E}}(\hat{f}_n)}_{\text{upper bound}} + f(\mathcal{H}, n)$$

dove il termine $f(\mathcal{H}, n)$ dipende dalla classe $\mathcal{H}$ di funzioni utilizzata e dal numero n di campioni.

Probabilità di avere un errore grande. Un modo per analizzare il comportamento dell'algoritmo è studiare la probabilità che l'excess risk sia grande. Per ogni $\varepsilon > 0$ vogliamo che

$$\mathbb{P}\left[\mathcal{E}(\hat{f}_n) - \min_f \mathcal{E}(f) > \varepsilon\right] \leq \delta(\varepsilon, n)$$

dove $\delta(\varepsilon, n)$ si chiama **rate** e descrive come l'errore dipende dal numero di campioni n e l'accuratezza ε. In generale vogliamo che aumentando il numero di dati la probabilità di fare un errore grande diminuisca.

Consistency. Un'altra proprietà importante è la consistency. Un algoritmo è consistente se, quando il numero di dati tende all'infinito, la probabilità di avere un errore maggiore di ε tende a 0:

$$\lim_{n\to\infty} \mathbb{P}\left[\mathcal{E}(\hat{f}_n) - \min_f \mathcal{E}(f) > \varepsilon\right] = 0$$

Se questa proprietà vale per qualsiasi distribuzione $p(x, y)$, allora il metodo è detto universalmente consistente. In generale la consistency è più facile da ottenere quando lo spazio degli input X è finito o enumerabile oppure quando esiste una relazione continua tra input e output. Un esempio classico di algoritmo consistente è K-NN (K Nearest Neighbours). Questo algoritmo è semplice perchè memorizza tutti i dati di training e quando arriva un nuovo punto x, guarda i punti di training più vicini. La vicinanza viene definita tramite una metrica. Se $k = 1$ l'algoritmo guarda solo il punto più vicino e assegna la stessa etichetta; se $1 < k < n$ si prendono i k punti più vicini e si assegna la classe più frequente tra questi; se $k = n$ cioè tutti i dati, il modello predice semplicemente la classe più frequente nel dataset e questo classificatore si chiama **base constant classifier** perchè ignora completamente la struttura geometrica dei dati e guarda solo la distribuzione delle etichette (pertanto è una classificazione molto debole). Il metodo KNN è consistente e soddisfa

$$\text{err}_{KNN} \leq 2\text{err}_{Bayes}$$

1.3 Bayes optimal estimator

Il Bayes optimal estimator è il classificatore teoricamente migliore possibile. Nel caso di classificazione binaria $y \in \{0, 1\}$ definiamo $\eta(x) = \mathbb{E}[Y|X = x]$ che rappresenta la probabilità condizionata della classe. Il classificatore ottimale è

$$f(x) = \mathbf{1}\{\eta(x) \geq 1/2\}$$

cioè assegna la classe 1 se la probabilità è maggiore di $1/2$ altrimenti assegna 0.

Caso regressione. Nel caso della regressione il predittore ottimale è semplicemente

$$f(x) = \eta(x) = \mathbb{E}[Y|X = x]$$

cioè il valore atteso dell'output dato l'input.

1.4 Scelta della classe di funzioni

Quando costruiamo un modello dobbiamo scegliere una classe di funzioni $h \in \mathcal{H}$ da cui selezionare il nostro modello. Ci sono due casi estremi:

- Se $\mathcal{H}$ è troppo grande: può approssimare bene la vera funzione ma rischiamo l'overfitting.
- Se $\mathcal{H}$ è troppo piccola: il modello è troppo semplice e non riuscirà a rappresentare la funzione reale (underfitting).

La soluzione è usare una classe grande ma aggiungere una regolarizzazione. Si minimizza

$$L\left(f_\theta(x), y\right) + \lambda ||\theta||_2^2$$

cioè loss sui dati più una penalizzazione sulla complessità del modello (principio ERM + regularization).

1.5 No free lunch theorem

Il no free lunch theorem dice che non esiste un algoritmo di apprendimento che sia sempre migliore degli altri per ogni possibile distribuzione di dati. Formalmente:

$$\forall \hat{f}_n, \forall n, \forall \varepsilon > 0 \quad \exists\, p(x,y) : \mathcal{E}(f) = 0 \quad \text{ma} \quad \mathcal{E}(\hat{f}_n) \geq \frac{1}{2} - \varepsilon$$

In altre parole: senza fare assunzioni sui dati nessun algoritmo può funzionare bene in generale. Per questo è necessario fare delle ipotesi, ad esempio:

- imporre restrizioni sulla distribuzione $p(x, y)$
- limitare la classe di funzioni $h \in \mathcal{H}$

1.6 Modelli generativi e discriminativi

Possiamo scomporre la distribuzione congiunta di probabilità $p(x, y)$ come segue

$$p(x,y) = p(x)p(y|x) = p(y)p(x|y)$$

questo porta a due tipi di modelli:

- Modelli generativi che stimano la distribuzione completa dei dati, ad esempio $p(x|y)$ spesso assumendo che sia gaussiana (ad esempio nei Gaussian Mixture Models o nel Gaussian Discriminant Analysis).
- Modelli discriminativi che stimano la relazione tra input e output, cioè $p(x|y)$ oppure direttamente la funzione $f(x)$ (ad esempio linear regression, logistic regression e reti neurali).

1.7 PAC Learning

Diverse domande fondamentali emergono quando progettiamo e analizziamo un algoritmo che apprende dagli esempi, come ad esempio in che modo possiamo ottimizzare l'apprendimento o quanti camponi ci servono per ottenere una buona soluzione del nostro problema. Per provare a rispondere a queste domande introduciamo PAC learning (probably approximately correct learning). Definiamo X l'input space e Y il set di etichette. Un concetto $c \in C$ dove C è il concept-space, è una funzione $c : X \to Y$ che mappa elementi di X a Y. Assumiamo che gli esempi siano indipendenti e identicamente distribuiti (i.i.d.) e che seguano una distribuzione di probabilità D. Il learner considera un insieme di ipotesi H (che potrebbe non coincidere con C) e riceve un sample $S = (x_1, ..., x_m)$ con le etichette $(c(x_1), ..., c(x_m))$, basate su uno specifico concept c da apprendere. Il suo scopo è quello di usare S per selezionare un'ipotesi $h_s \in H$ che presenta un errore di generalizzazione $\mathcal{E}(h)$ piccolo rispetto al concept c.

Data un'ipotesi $h \in H$, un concetto $c \in C$ e D distribuzione di probabilità, definiamo l'errore di generalizzazione o risk di h come

$$\mathcal{E}(h) = \mathbb{P}[h(x) \neq c(x)] = \mathbb{E}[1_{h(x)\neq c(x)}], \qquad x \sim D.$$

L'errore di generalizzazione relativo ad $h \in H$ non è accessibile direttamente dal learner in quanto D e c sono ignoti. Tuttavia il learner può stimare l'errore empirico di un'ipotesi sul sample.

Data un'ipotesi $h \in H$, un concetto $c \in C$ e un sample $S = (x_1, ..., x_m)$, l'errore empirico (o empirical risk) di h è definito da

$$\hat{\mathcal{E}}(h) = \frac{1}{m} \sum_{i=1}^{m} 1_{h(x_i)\neq c(x_i)}$$

Quindi $\hat{\mathcal{E}}(h)$ è l'errore medio sul sample S, mentre l'errore di generalizzazione è l'errore atteso basato sulla distribuzione di probabilità D. Per un $h \in H$ fissato, il valore atteso dell'errore empirico basato su un sample S con distribuzione i.i.d. è uguale all'errore di generalizzazione, infatti sfruttando la proprietà di linearità dell'aspettazione possiamo scrivere

$$E[\hat{\mathcal{E}}(h)] = \frac{1}{m} \sum_{i=1}^{m} \mathbb{E}[1_{h(x_i)\neq c(x_i)}] = \frac{1}{m} \sum_{i=1}^{m} \mathbb{E}[1_{h(x)\neq c(x)}] = \mathbb{E}[1_{h(x)\neq c(x)}] = \mathcal{E}(h).$$

1.7.1 PAC Learnability

Una concept class C è detta PAC-learnable se esiste un algoritmo A e una funzione polinomiale $poly(\cdot, \cdot, \cdot, \cdot)$ tale che $\forall \varepsilon > 0, \delta > 0$, per ogni distribuzione D su X e per ogni target-concept $c \in C$, vale la seguente relazione per ogni sample di dimensione $m \geq poly(1/\varepsilon, 1/\delta, n, size(c))$

$$\mathbb{P}[\mathcal{E}(h_s) \leq \varepsilon] \geq 1 - \delta.$$

Se l'algoritmo A viene eseguito in tempo polinomiale come definito sopra, allora C è detto efficientemente PAC learnable; e quando tale algoritmo A esiste viene chiamato PAC-learning algorithm per C. Una concept-class C è PAC learnable se dopo aver osservato un numero di punti polinomiale in $1/\varepsilon$ e $1/\delta$ è approssimativamente corretto (errore al più ε) con alta probabilità (almeno $1-\delta$). Quindi $1-\delta$ è la confidenza e $1-\varepsilon$ l'accuratezza.

1.8 Consistent learner

Introduciamo ora un generalization bound per ipotesi consistenti. Innanzitutto un learner è detto consistent se riesce a trovare un'ipotesi che classifica correttamente tutti gli esempi del training set. Formalmente quindi avremmo che

$$h(x_i) = c(x_i) \quad \forall i.$$

Questo significa che l'errore sul training set $\hat{\mathcal{E}}(h) = 0$, e quindi il modello è perfettamente consistente con i dati osservati.

Consideriamo il caso in cui lo spazio delle ipotesi H è finito, ovvero la sua cardinalità $|H|$ è finita. Assumiamo che le ipotesi siano consistenti e che quindi il target concept sia nello spazio delle ipotesi stesso, $c \in H$.

Teorema (Learning bound - finite case). Sia H un insieme di funzioni che mappano da X a Y. Sia A un algoritmo che per ogni target $c \in H$ e un i.i.d. sample S ritorna un'ipotesi consistente $h_s : \hat{\mathcal{E}}(h_s) = 0$. Allora per ogni $\varepsilon, \delta > 0$ la disuguaglianza $\mathbb{P}[\mathcal{E}(h_s) \leq \varepsilon] \geq 1 - \delta$ è vera se

$$m \geq \frac{1}{\varepsilon}\left(\log|H| + \log\frac{1}{\delta}\right)$$

Dimostrazione. L'idea è stimare la probabilità che esista un'ipotesi con errore di generalizzazione maggiore di ε, che risulti comunque consistente con tutti gli esempi del training set. Indichiamo con $\bar{H}$ l'insieme di queste ipotesi. Se $h \in \bar{H}$, allora la probabilità che classifichi correttamente un singolo esempio estratto da $p(x)$ è al massimo $1-\varepsilon$, di conseguenza, la probabilità che la stessa ipotesi sia consistente con m esempi indipendenti è al più $(1-\varepsilon)^m$. Utilizzando la disuguaglianza $(1-\varepsilon)^m \leq e^{-\varepsilon m}$ e sapendo che $|\bar{H}| \leq |H|$ si ottiene

$$\mathbb{P}\left[\hat{\mathcal{E}}(h) = 0 | \mathcal{E}(h) > \varepsilon\right] \leq \sum_{h \in \bar{H}} e^{-\varepsilon m} \leq |\bar{H}|\, e^{-\varepsilon m} \leq |H|\, e^{-\varepsilon m}$$

Imponendo che questa probabilità sia minore o uguale a δ e risolvendo rispetto a m otteniamo

$$|H|e^{-\varepsilon m} \leq \delta \qquad \Longleftrightarrow \qquad m \geq \frac{1}{\varepsilon}\left(\log|H| + \log\frac{1}{\delta}\right)$$

□

1.9 Inconsistent learner

Nel caso precedente abbiamo considerato un *consistent learner*, cioè un algoritmo che riesce a trovare un'ipotesi h che classifica correttamente tutti gli esempi del training set, quindi con errore empirico nullo.

Tuttavia, in molti casi pratici questo non è possibile. Un learner è detto *inconsistent* quando non esiste alcuna ipotesi nello spazio $\mathcal{H}$ che riesca a classificare perfettamente tutti gli esempi di training. Questo può accadere, ad esempio, quando la funzione target reale non appartiene allo spazio delle ipotesi considerato, cioè quando

$$\nexists\, h \in \mathcal{H} \text{ tale che } h(x_i) = c(x_i) \quad \forall i.$$

In questa situazione il learner non può ottenere errore empirico nullo. L'obiettivo dell'algoritmo diventa quindi trovare un'ipotesi che minimizzi l'errore sul training set. In questo caso non possiamo più garantire che l'errore empirico sia zero, ma vogliamo comunque controllare la differenza tra l'errore vero (errore di generalizzazione) e l'errore empirico. In particolare vogliamo garantire che

$$\mathcal{E}(h) \leq \hat{\mathcal{E}}(h) + \varepsilon$$

cioè che l'errore vero non sia molto più grande dell'errore osservato sul training set. Più precisamente, vogliamo che con alta probabilità valga

$$\mathbb{P}[\mathcal{E}(h) \leq \hat{\mathcal{E}}(h) + \varepsilon] \geq 1 - \delta.$$

Per studiare questa probabilità possiamo utilizzare *Hoeffding bound* (che deriva dalla legge dei grandi numeri). Questo permette di limitare la probabilità che l'errore empirico differisca troppo dall'errore vero. In particolare otteniamo

$$\mathbb{P}[\mathcal{E}(h) \geq \hat{\mathcal{E}}(h) + \varepsilon] < e^{-2m\varepsilon^2}.$$

Qui $\hat{\mathcal{E}}(h)$ è una quantità aleatoria perché dipende dal campione di training osservato. Questa disuguaglianza rappresenta la probabilità che l'errore vero sia significativamente più grande dell'errore empirico, cioè la probabilità di avere un'ipotesi il cui errore sul training set sia fuorviante rispetto all'errore reale.

Tuttavia questa stima vale per una singola ipotesi h. Poiché lo spazio delle ipotesi contiene più funzioni, dobbiamo considerare la probabilità che esista almeno una ipotesi con questa proprietà. Utilizzando lo *union bound* otteniamo

$$\mathbb{P}[\exists h \in \mathcal{H} : \mathcal{E}(h) \geq \hat{\mathcal{E}}(h) + \varepsilon] \leq |\mathcal{H}| \cdot e^{-2m\varepsilon^2}.$$

Vogliamo che questa probabilità sia minore o uguale a δ, quindi imponiamo

$$\delta \leq |\mathcal{H}| \cdot e^{-2m\varepsilon^2}.$$

Risolvendo rispetto a m (il numero di esempi di training) otteniamo il numero minimo di campioni necessari:

$$m \geq \frac{1}{2\varepsilon^2}\left[\log|\mathcal{H}| + \log\frac{1}{\delta}\right]$$

che rappresenta la *sample complexity* nel caso di learner non consistent.

1.9.1 Agnostic PAC learning

Sia H un set di ipotesi. A è un algoritmo di agnostic PAC learning se esiste una funzione polinomiale $poly(\cdot,\cdot,\cdot,\cdot)$ tale che $\forall \varepsilon, \delta > 0$ e per ogni distribuzione D su $X \times Y$, vale che per ogni sample di grandezza $m \geq poly\left(1/\varepsilon, 1/\delta, n, size(c)\right)$:

$$\mathbb{P}\left[\mathcal{E}(h_s) - \min_{h\in H}\mathcal{E}(h) \leq \varepsilon\right] \geq 1 - \delta$$

Se A viene eseguito in tempo polinomiale quindi diremo che è un efficient agnostic PAC learning algorithm.

1.10 Infinite class

1.10.1 Infinite class e shattering

Se lo spazio delle ipotesi H è infinito, non è possibile applicare direttamente risultati che dipendono dalla cardinalità (come nel caso finito, dove compare $\log|H|$. È quindi necessario introdurre una nozione di **complessità** dello spazio delle ipotesi che non dipenda semplicemente dal numero di elementi, ma dalla loro capacità espressiva. L'idea è quella di misurare quanto bene le ipotesi in H riescano a discriminare sottoinsiemi finiti dello spazio di input X. Dato un insieme finito $S = \{x_i, ..., x_n\} \subseteq X$, una **dicotomia** su S è una partizione degli elementi in due classi $(+,-)$; formalmente suddividiamo in

$$P \subseteq S, \qquad S \setminus P$$

ogni dicotomia rappresenta un possibile assegnamento di etichette binarie agli elementi di S. Da ciò possiamo dedurre che il numero totale di dicotomie in un insieme di cardinalità m è di 2^m. Ora definiamo il concetto di **shattering**:

$$S \text{ è shattered da } H \quad \iff \quad \forall \text{ dicotomia su } S, \exists h \in H : h \text{ consistent su } S$$

dove consistent significa che $h(x)$ è uguale all'etichetta assegnata dalla dicotomia $\forall x \in S$.

1.10.2 VC-Dimension

Se lo spazio delle ipotesi $\mathcal{H}$ è infinito non posso enumerare il set, quindi dobbiamo introdurre la complessità dello spazio delle ipotesi. Ci sono vari modi per farlo: l'idea è

andare nello spazio di input e si guardano i numeri di istanze dell'input che possono essere completamente discriminati da H; questa misura prende il nome di VC-dimension (viene da Vapnick-Chervonenkis). La VC-dimension di H su X è la grandezza del più grande subset finito di X shattered da $\mathcal{H}$. La VC-dimension può essere, in alcune classi, infinita. In altre parole, è il massimo numero di punti che $\mathcal{H}$ può classificare in tutti i modi possibili. Per i classificatori lineari in $\mathbb{R}^d$ si ha che

$$VC = d + 1,$$

cioè la VC-dimension è la dimensione dello spazio più uno, perché si può trovare un insieme di $d + 1$ punti che può essere shattered, ma non esiste un insieme di $d + 2$ punti che può esserlo. Se invece esistono insiemi di dimensione arbitrariamente grande che possono essere shattered da $\mathcal{H}$, allora la VC-dimension è infinita.

Un'altra osservazione utile riguarda la relazione tra VC-dimension e numero di ipotesi: se $\mathcal{H}$ ha VC-dimension d, allora per poter shatterare d punti, $\mathcal{H}$ deve contenere almeno 2^d ipotesi, cioè

$$2^d \leq |\mathcal{H}| \quad \implies \quad VC(\mathcal{H}) = d \leq \log_2 |\mathcal{H}|.$$

Questa relazione è importante perché ci aiuta a capire come la complessità della classe di ipotesi vincoli la capacità di rappresentare combinazioni di etichette sui punti dell'input.

2 Principal Component Analysis

2.1 Richiami di algebra lineare

2.1.1 Matrici come mappe lineari

In machine learning è utile interpretare le matrici come **trasformazioni lineari**, cioè funzioni che mappano uno spazio vettoriale in un altro: data una matrice $A \in \mathbb{R}^{m \times n}$, essa definisce una funzione $A : \mathbb{R}^n \to \mathbb{R}^m$, che può essere vista come una funzione parametrizzata che agisce sui dati. Dal punto di vista geometrico, una trasformazione lineare può ruotare, scalare (allungare o comprimere) e inclinare lo spazio, ma non può traslarlo; applicare una matrice a un insieme di punti significa quindi modificarne la struttura geometrica, alterando distanze, angoli e volumi. Questa operazione può introdurre una **distorsione dello spazio**, soprattutto quando la trasformazione agisce in modo diverso lungo direzioni diverse: tali direzioni sono descritte dagli autovettori della matrice, mentre gli autovalori indicano quanto ogni direzione viene espansa o compressa (autovalori grandi $\to$ espansione, piccoli $\to$ compressione, nulli $\to$ collasso). Un caso particolarmente rilevante si verifica quando la matrice non ha rango pieno, cioè quando le sue colonne non sono linearmente indipendenti ($\text{rank}(A) < n$): in questo caso la trasformazione non è iniettiva, più punti distinti nello spazio di input vengono mappati nello stesso punto e quindi il mapping non è invertibile, con conseguente perdita di informazione. Geometricamente, ciò corrisponde a uno "schiacciamento" dello spazio su un sottospazio di dimensione inferiore (ad esempio da $\mathbb{R}^3$ a un piano o a una retta), rappresentando un

caso estremo di distorsione. Questi fenomeni si estendono naturalmente anche a spazi ad alta dimensione, dove le matrici possono amplificare alcune direzioni e comprimere altre, determinando la quantità e la qualità dell'informazione preservata; per questo motivo, in molti algoritmi di machine learning, proprietà come il rango e la distribuzione degli autovalori (o valori singolari) giocano un ruolo cruciale nel determinare stabilità numerica, capacità di rappresentazione e prestazioni del modello. Per essere una linear map devono valere le seguenti condizioni:

1. $F(v_1 + v_2) = F(v_1) + F(v_2)$
2. $F(\alpha v) = \alpha\, F(v)$

Tutto ciò che viene trasformato da una matrice può essere espresso come **somma ponderata delle colonne della matrice stessa**: infatti, data $X \in \mathbb{R}^{m\times n}$ e $x \in \mathbb{R}^n$, possiamo scrivere il prodotto Xx come combinazione lineare delle colonne di X, cioè $Xx = \sum_{i=1}^{n} x_i X_i$, dove X_i è la i-esima colonna di X. Una collezione di vettori $v_1, ..., v_n$ si dice linearmente indipendente se l'unica combinazione lineare che produce il vettore nullo è quella banale (tutti i coefficienti α_i nulli); viceversa, se esistono coefficienti non tutti nulli che soddisfano tale uguaglianza, i vettori sono **linearmente dipendenti**. Nel contesto delle matrici, una dipendenza lineare tra le colonne indica che almeno una colonna può essere espressa come combinazione lineare delle altre, e ciò implica che le colonne non generano direzioni "nuove" nello spazio: geometricamente, la matrice sta comprimendo lo spazio di partenza in un sottospazio di dimensione inferiore. Se invece le colonne di una matrice X sono linearmente indipendenti, allora non vi è perdita di informazione lungo le direzioni considerate e la trasformazione è iniettiva (nel caso $m \geq n$), in particolare se X è una matrice quadrata $n \times n$, con colonne indipendenti, allora è invertibile e ammette una inversa X^{-1}, e significa che la trasformazione può essere completamente invertita senza perdita di informazione. Se la matrice è invertibile allora $AA^{-1} = A^{-1}A = \mathbb{I}$.

Un'altra quantità fondamentale associata a una matrice quadrata è il **determinante**, che fornisce importanti informazioni geometriche sulla trasformazione lineare indotta dalla matrice. Nel caso di una matrice 2×2, il determinante può essere interpretato come l'**area orientata del parallelogramma** generato dai vettori colonna della matrice; più in generale, il valore assoluto del determinante misura il **fattore di scala** con cui la trasformazione modifica aree (in 2D), volumi (in 3D) o, in dimensione n, l'**ipervolume**. In particolare, se $\det A = 0$, la trasformazione comprime lo spazio in un sottospazio di dimensione inferiore, implicando una perdita di informazione e la non invertibilità della matrice; se $|\det A| > 0$, la trasformazione preserva la dimensione dello spazio e risulta invertibile. Inoltre, il segno del determinante ha un significato geometrico: se $\det A > 0$, la trasformazione preserva l'orientazione dello spazio, mentre se $\det A < 0$, oltre a scalare lo spazio, ne inverte l'orientazione (ad esempio producendo una riflessione). Pertanto, il determinante riassume in un'unica quantità sia l'effetto di **scala** sia l'eventuale **ribaltamento** introdotto dalla trasformazione lineare.

Supponiamo di avere due vettori: v e un vettore colonna w. Vogliamo proiettare v su w o meglio proiettare v nel sottospazio di w. Sfruttando le proprietà di trigonometria

sappiamo che $||v|| \cos\theta$ è la lunghezza della proiezione del vettore v nella direzione w. Per proiettare un vettore v nello spazio definito da w possiamo scrivere la proiezione come

$$P_w(v) = \frac{w\, w^\top}{w^\top w} v = \left(\frac{w}{||w||}\right)\left(\frac{w}{||w||}\right)^\top v$$

definendo il versore $\hat{w}$ otteniamo che

$$P_w(v) = \hat{w}(\hat{w}^\top v)$$

Supponiamo di avere due vettori $v \in \mathbb{R}^n$ e un vettore colonna $w \in \mathbb{R}^n$ e di voler proiettare v nel sottospazio generato da w. Dal punto di vista geometrico, questa operazione consiste nel trovare la componente di v lungo la direzione di w. Dalla trigonometria sappiamo che la lunghezza della proiezione di v nella direzione di w è data da $||v|| \cos\theta$, dove θ è l'angolo tra i due vettori. Poichè il coseno dell'angolo può essere espresso tramite il prodotto scalare (**cosine similarity**), cioè

$$\cos\theta = \frac{v^\top w}{||v||\,||w||}$$

si ottiene che la proiezione dipende direttamente dal prodotto scalare tra i due vettori, che è alla base anche della **cosine similarity**. Formalmente, la proiezione di v sul sottospazio generato da w si può scrivere come

$$P_w(v) = \frac{w\, w^\top}{w^\top w} v = \left(\frac{w}{||w||}\right)\left(\frac{w}{||w||}\right)^\top v$$

dove $\frac{ww^\top}{w^\top w}$ è la matrice di proiezione sul sottospazio w. Normalizzando w e introducendo il versore $\hat{w}$ otteniamo che

$$P_w(v) = \hat{w}(\hat{w}^\top v).$$

In questa forma è particolarmente evidente che il termine $\hat{w}^\top v$ rappresenta la componente scalare di v lungo w, cioè quanto v è allineato con w, ed è proporzionale alla cosine similarity tra i due vettori; moltiplicando poi per $\hat{w}$ si ottiene il vettore proiettato nella direzione di w.

2.1.2 Autovalori e autovettori

Data una matrice quadrata $A \in \mathbb{R}^{n\times n}$, diremo che $\lambda \in \mathbb{C}$ è un autovalore di A e $x \in \mathbb{C}^n$ è l'autovettore corrispondente se

$$Ax = \lambda x, \quad x \neq 0$$

Geometricamente significa che moltiplicare A per il vettore x restituisce nuovo vettore nella stessa direzione di x, riscalato secondo λ. Per calcolare gli autovalori della matrice si risolve per λ l'equazione

$$\det(A - \lambda\mathbb{I}) = 0$$

Dopo aver trovato gli autovalori, possiamo ricavare gli autovettori corrispondenti risolvendo l'equazione $Ax = \lambda x$.

2.1.3 Eigendecomposition

Dopo aver trovato autovalori e autovettori, ci occupiamo di eigendecomposition di una matrice A. Definiamo le matrici W e Σ come segue:

$$W = \begin{bmatrix} | & & | \\ x_{\lambda_1} & \dots & x_{\lambda_n} \\ | & & | \end{bmatrix}, \qquad \Sigma = \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{bmatrix}$$

La matrice W è la matrice che ha per colonne gli autovettori della matrice A, mentre Σ è la matriche che ha gli autovalori corrispondenti sulla diagonale. Per definizione di autovalori e autovettori abbiamo che

$$AW = W\Sigma$$

La matrice W è invertibile (esiste W^{-1}), quindi moltiplichiamo ambo i membri a destra per W^{-1}, trovando che

$$A = W\Sigma W^{-1}$$

Una buona proprietà dell'eigendecomposition è che possiamo scrivere diverse operazioni che incontriamo in termini dell'eigendecomposition. Come primo esempio possiamo considerare

$$A^n = \underbrace{A \cdots A}_{n \text{ volte}} = \underbrace{(W\Sigma W^{-1}) \cdots (W\Sigma W^{-1})}_{n \text{ volte}} = W \underbrace{\Sigma \cdots \Sigma}_{n \text{ volte}} W^{-1} = W\Sigma^n W^{-1}$$

Con questo notiamo che per ogni potenza della matrice, l'eigendecomposition è ottenuta semplicemente elevando gli autovalori alla stessa potenza. Analogamente con esponente negativo otteniamo

$$A^{-1} = W\Sigma^{-1}W^{-1}$$

2.1.4 Teorema Spettrale

Sia $A \in \mathbb{R}^{n \times n}$ una matrice quadrata e simmetrica, cioè tale che $A = A^\top$. In questo caso, il teorema spettrale garantisce che tutti gli autovalori di A sono reali e che esiste una base ortonormale di $\mathbb{R}^n$ composta da autovettori della matrice; in particolare, autovettori associati ad autovalori distinti risultano ortogonali tra loro.

Di conseguenza, la matrice A è diagonalizzabile tramite una matrice ortogonale e ammette una decomposizione spettrale della forma $A = U\Sigma U^\top$, dove U è una matrice ortogonale le cui colonne sono autovettori ortonormali di A e Σ è una matrice diagonale contenente gli autovalori. Questa rappresentazione è un caso particolare della più generale eigendecomposition $U\Sigma U^{-1}$, che nel caso simmetrico si semplifica grazie al fatto che $U^{-1} = U^\top$. Inoltre, la Singular Value Decomposition (SVD), che in generale si scrive $A = U\Sigma V^\top$, coincide con la decomposizione spettrale quando A è simmetrica, poiché in tal caso vale $U = V$.

Dal punto di vista geometrico, l'azione della matrice su un vettore $x \in \mathbb{R}^n$ può essere espressa come $Ax = U(\Sigma(U^{-1}x))$, evidenziando una trasformazione composta da tre fasi: una prima rotazione (o cambio di base) che proietta il vettore nella base degli autovettori, una scalatura lungo tali direzioni determinata dagli autovalori e infine una rotazione inversa che riporta il vettore nello spazio originale (rotate–scale–rotate).

2.2 PCA

La Principal Component Analysis (PCA) è una tecnica fondamentale nell'ambito del machine learning e della statistica multivariata, utilizzata per analizzare la struttura dei dati in assenza di etichette, quindi in un contesto di apprendimento non supervisionato. L'idea di base è quella di rappresentare un insieme di dati, inizialmente descritto in uno spazio ad alta dimensionalità, in uno spazio di dimensione inferiore, preservando il più possibile l'informazione contenuta nei dati. Tale informazione viene formalizzata in termini di varianza: la PCA cerca infatti le direzioni lungo le quali i dati variano di più.

La Principal Component Analysis (PCA) è una tecnica di apprendimento non supervisionato utilizzata per l'analisi e la riduzione della dimensionalità dei dati. Si consideri un insieme di dati $\{x_i\}_{i=1}^N \sim \mathcal{D}$, dove la distribuzione $\mathcal{D}$ è ignota e ciascun dato $x_i \in \mathbb{R}^D$. Si assume tipicamente che $N \geq D$, ovvero che il numero di osservazioni sia almeno pari (o superiore) alla dimensionalità dello spazio delle caratteristiche.

L'obiettivo della PCA è individuare una trasformazione lineare che consenta di rappresentare i dati in uno spazio di dimensione inferiore, preservando il più possibile l'informazione contenuta nei dati originali. In particolare, si cerca un sottospazio di dimensione ridotta tale che la proiezione dei dati su di esso massimizzi la loro dispersione (varianza).

Nel caso più semplice, si considera la ricerca di una singola direzione $u \in \mathbb{R}^D$, con vincolo di normalizzazione $||u||_2 = 1$, lungo la quale proiettare i dati. La proiezione di un punto x lungo tale direzione è data da:

$$x' = P_u x = (x^\top u)u$$

Il problema della PCA consiste quindi nel determinare il vettore u che massimizza la varianza dei dati proiettati. Equivalentemente, si vuole massimizzare la varianza delle proiezioni scalari dei dati lungo la direzione u. Questo problema può essere riformulato ricorrendo alla norma euclidea L_2 della proiezione vettoriale. Infatti la proiezione di x_i lungo la direzione u è data da $(x_i^\top u)u$ e si può considerare il problema di ottimizzazione

$$\arg\min_u \frac{1}{N} \sum_{i=1}^{N} ||(x_i^\top u)u||_2^2$$

che evidenzia la ricerca della direzione lungo la quale la norma media delle proiezioni dei dati risulti massima [Figura 1] (equivale a massimizzare la varianza).

Generalizzando al caso multidimensionale, si cercano k direzioni ortonormali raccolte nelle colonne di una matrice $U \in \mathbb{R}^{D \times k}$, tali che la proiezione dei dati nello spazio ridotto

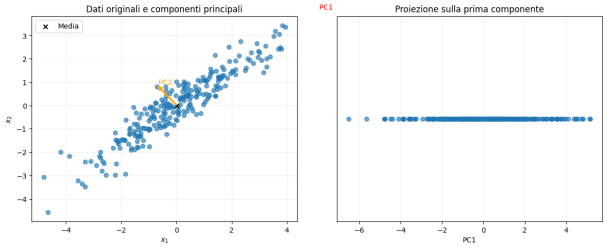


Figura 1: Visualizzazione della PCA

sia data da:

$$x' = U^\top x$$

e tali direzioni massimizzino la varianza totale dei dati proiettati. La soluzione di questo problema è strettamente legata alla decomposizione spettrale della matrice di covarianza dei dati: le direzioni ottimali corrispondono infatti agli autovettori associati ai maggiori autovalori della matrice di covarianza. In questo modo, la PCA individua le direzioni di massima variabilità dei dati, fornendo una rappresentazione compatta ed efficiente degli stessi.

2.2.1 Matrice di covarianza

Si consideri un insieme di dati rappresentato da una matrice $X \in \mathbb{R}^{N \times D}$, dove $N \geq D$ è il numero di campioni e D il numero di feature. Sia inoltre $k \leq D$ la dimensione del sottospazio in cui si desidera proiettare i dati. In questo contesto, la matrice di covarianza appartiene a $\mathbb{R}^{D \times D}$.

Indicando con $\mu \in \mathbb{R}^D$ il vettore media del dataset, la matrice di covarianza $C \in \mathbb{R}^{D \times D}$ è definita come:

$$C = \frac{1}{N}(X - \mu)^\top (X - \mu).$$

Assumendo che i dati siano campionati da una distribuzione gaussiana multivariata, è pratica comune effettuare una fase di preprocessing che consiste nel centraggio dei dati (zero-mean) e, opzionalmente, nella loro standardizzazione:

$$X' = \frac{X - \mu}{\sigma},$$

dove σ rappresenta la deviazione standard per ciascuna feature. In questo modo si ottiene un dataset con media nulla e varianza unitaria.

L'obiettivo della Principal Component Analysis (PCA) è quello di trovare una trasformazione lineare che consenta di ridurre la dimensionalità dei dati preservando il più possibile l'informazione. In particolare, si cerca un sottospazio k-dimensionale che massimizzi la varianza dei dati proiettati.

Nel caso di una singola direzione, si cerca un vettore $u \in \mathbb{R}^D$ tale che $\|u\|_2 = 1$. La proiezione di un punto x lungo tale direzione è:

$$x' = P_u x = \frac{uu^\top}{u^\top u} x = (x^\top u)u.$$

Il problema consiste nel massimizzare la varianza dei dati proiettati, che può essere espressa tramite la norma L_2^2:

$$\arg\max_u \frac{1}{N} \sum_{i=1}^{N} \|P_u x_i\|_2^2 = \arg\max_u \frac{1}{N} \sum_{i=1}^{N} \|(x_i^\top u)u\|_2^2.$$

Sfruttando il vincolo $\|u\|_2 = 1$, si ottiene:

$$\frac{1}{N} \sum_{i=1}^{N} \|(x_i^\top u)u\|_2^2 = \frac{1}{N} \sum_{i=1}^{N} (x_i^\top u)^2 = \frac{1}{N} \sum_{i=1}^{N} u^\top x_i x_i^\top u.$$

Pertanto, il problema si riconduce alla forma:

$$\arg\max_u \; u^\top C u, \quad \text{con } C = \frac{1}{N} X^\top X,$$

dove si assume che i dati siano già centrati (zero-mean).
La soluzione di questo problema è data dall'autovettore della matrice di covarianza C associato al massimo autovalore. Più in generale, indicando con

$$(\lambda_1, u_1), \ldots, (\lambda_D, u_D) = \text{eig}\left(\frac{1}{N} X^\top X\right),$$

gli autovalori (ordinati in senso decrescente) e i corrispondenti autovettori, le direzioni principali della PCA sono date dagli autovettori associati ai maggiori autovalori.

Per ridurre la dimensionalità da D a k, si selezionano i primi k autovettori, scegliendo k in modo da preservare la maggior parte della varianza (energia) dei dati:

$$\frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{D} \lambda_i} \geq 0.95.$$

Se invece dovessimo proiettare i dati in più dimensioni, allora avremo che

$$C = U \Sigma U^\top = \frac{1}{N} X X^\top$$

Osservazione: nel caso monodimensionale si considera un vettore u, mentre nel caso multidimensionale si introduce una matrice U le cui colonne sono gli autovettori principali; questa matrice definisce il sottospazio su cui i dati vengono proiettati

2.2.2 Rotazione e decorrelazione tramite PCA

Un'importante applicazione della PCA consiste nella rotazione e decorrelazione dei dati, senza necessariamente effettuare una riduzione della dimensionalità. Si consideri il dataset standardizzato:

$$\bar{X} = \frac{X - \mu}{\sigma},$$

dove i dati sono stati centrati (media nulla) e scalati (varianza unitaria per ciascuna feature).

Sia $U \in \mathbb{R}^{D \times D}$ la matrice ortogonale i cui vettori colonna sono gli autovettori della matrice di covarianza. La trasformazione PCA dei dati è quindi data da:

$$\bar{x}_p = U^\top \bar{x}.$$

Questa operazione corrisponde a una rotazione del sistema di riferimento: il vettore $\bar{x}$ viene espresso nella base degli autovettori della matrice di covarianza.

Nel nuovo sistema di coordinate, le componenti di $\bar{x}_p$ risultano non correlate, poiché la matrice di covarianza dei dati trasformati è diagonale. In particolare, tale trasformazione consente di decorrelare le feature mantenendo invariata la dimensionalità del problema.

2.2.3 Whitening (sferizzazione) dei dati

Supponiamo di avere il dataset standardizzato

$$\bar{X} = \frac{X - \mu}{\sigma},$$

dove i dati sono centrati e scalati. Se applichiamo la **whitening** (o sferizzazione) tramite PCA, otteniamo:

$$\bar{x}_p^\top = \Sigma^{-1/2} U^\top \bar{x}^\top,$$

dove:

- U è la matrice ortogonale i cui vettori colonna sono gli autovettori della matrice di covarianza di $\bar{X}$,
- $\Sigma = \text{diag}(\lambda_1, \ldots, \lambda_D)$ contiene gli autovalori della matrice di covarianza di $\bar{X}$,
- $\Sigma^{-1/2} = \text{diag}(\lambda_1^{-1/2}, \ldots, \lambda_D^{-1/2})$.

In questo modo, otteniamo un dataset $\bar{X}_p$ i cui punti sono decorrelati e con varianza unitaria lungo ciascuna componente principale.

Infatti, la matrice di covarianza dei dati trasformati diventa:

$$\frac{1}{N} \bar{X}_p^\top \bar{X}_p = \frac{1}{N} (\Sigma^{-1/2} U^\top \bar{X}^\top)^\top (\Sigma^{-1/2} U^\top \bar{X}^\top) = \frac{1}{N} \bar{X} U \Sigma^{-1} U^\top \bar{X}^\top = \Sigma^{-1/2} U^\top C U \Sigma^{-1/2} = \mathbb{I},$$

dove C è la matrice di covarianza di $\bar{X}$. Questa trasformazione non solo decorrela i dati, ma li sferizza: ogni componente ha varianza 1 e tutte le componenti sono ortogonali tra loro. Tale procedura è utile in molti algoritmi di machine learning che assumono dati non correlati o con scala uniforme. Quindi diciamo che intuitivamente con PCA i dati vengono correlati con varianza diversa (diagonale con autovalori λ_i), con whitening invece la varianza viene normalizzata lungo ciascuna componente $\sqrt{\lambda_i}$.

Osservazione: in PCA gli autovettori rappresentano le direzioni degli assi della cloud ellittica che contiene i dati, mentre gli autovalori determinano la lunghezza dei suoi semiassi, $\sqrt{\lambda_i}$. Con sphering rendiamo l'ellisse una circonferenza (assi di lunghezza 1).

3 Gaussian Mixture Models

3.1 Distribuzione normale e mixture of gaussians

Consideriamo la distribuzione di probabilità gaussiana in una dimensione, dove μ è la media e σ^2 è la varianza. In particolare consideriamo la distribuzione standard con $\mu = 0$ e $\sigma = 1$

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}}\, e^{-\frac{(x-\mu)^2}{2\sigma^2}} \qquad \Longrightarrow \qquad f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

Nel caso di una gaussiana multivariata, la situazione è analoga al caso unidimensionale. Tuttavia, al posto della varianza scalare σ^2, compare una forma quadratica del tipo $x^\top Ax$. La distribuzione è data da

$$f(x) = \frac{1}{\sqrt{(2\pi)^k \det \Sigma}}\, e^{-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)}$$

dove Σ è la matrice di covarianza.

Supponiamo che Σ sia definita positiva. Nella formula compare il **determinante** di Σ, che rappresenta il fattore di scala degli ipervolumi associato alla trasformazione lineare: essendo il determinante il prodotto degli autovalori, esso descrive come cambia il volume.

Se la matrice **non è invertibile**, allora $\det \Sigma = 0$. Questo implica la presenza di almeno una componente principale con autovalore nullo. In tal caso si ha una singolarità: la varianza lungo quella direzione è nulla e la massa della distribuzione si concentra su un sottospazio a dimensione inferiore. L'obiettivo ora è fare fitting dei dati: mentre prima si partiva dalla distribuzione per generare dati, ora, dati i campioni, vogliamo ricostruire la distribuzione che li ha generati. Una possibile rappresentazione è

$$p(x) = \frac{1}{n} \sum_{i=1}^{n} \delta(x - x_i),$$

dove δ è la delta di Dirac, cioè una distribuzione concentrata in un punto (equivalente al limite di una gaussiana con varianza che tende a zero). Tuttavia, questo modello non è efficace perché porta a overfitting: si assegna tutta la massa esattamente sui punti osservati. Per questo motivo, si considerano modelli più flessibili. Ad esempio, se i dati presentano due mode, scegliere una sola gaussiana (ad esempio tramite maximum likelihood) porterebbe a underfitting. In tal caso, è più appropriato considerare una combinazione di gaussiane:

$$p(x) = \frac{1}{2}\mathcal{N}(x, \mu_1, \Sigma_1) + \frac{1}{2}\mathcal{N}(x, \mu_2, \Sigma_2).$$

Affinchè la somma di gaussiane sia una distribuzione di probabilità, è necessario **normalizzare** tramite opportuni coefficienti π_1, π_2 tali che

$$\int_X (\pi_1 \mathcal{N}(x, \pi_1, \Sigma_1) + \pi_2 \mathcal{N}(x, \pi_2, \Sigma_2))\, dx = 1$$

con $\pi_1 + \pi_2 = 1$ e $\pi_1, \pi_2 > 0$ (mixture coefficients). Generalizzando a n variabili gaussiane, otteniamo che la densità di probabilità diventa

$$p(x) = \sum_{i=1}^{n} \pi_i \mathcal{N}(x, \mu_i, \Sigma_i), \qquad \sum_{i=1}^{n} \pi_i = 1, \qquad \pi_i > 0 \quad \forall i.$$

3.2 Probabilità congiunta, marginale e condizionata

Indichiamo la distribuzione **congiunta** di due variabili aleatorie X e Y come $P(X, Y)$; essa descrive la densità delle due variabili considerate simultaneamente, ovvero la probabilità che si verifichino insieme.

La distribuzione congiunta può essere espressa in termini di probabilità condizionate. In particolare, possiamo scrivere:

$$P(X, Y) = P(X|Y)\, P(Y) = P(Y|X)\, P(X)$$

Queste relazioni permettono di collegare la probabilità congiunta alle probabilità marginali e condizionate.

A partire da tali espressioni, possiamo derivare il **teorema di Bayes**, che consente di invertire una probabilità condizionata:

$$P(X|Y) = P(Y|X)\, \frac{P(X)}{P(Y)}, \quad P(Y) \neq 0.$$

Nel caso in cui le variabili aleatorie X e Y siano indipendenti, la distribuzione congiunta si semplifica notevolmente e diventa il prodotto delle probabilità marginali:

$$P(X, Y) = P(X)\, P(Y).$$

3.3 Learning dei parametri

Supponiamo di avere un insieme di dati di training $X = \{x_1, \ldots, x_N\}$ in uno spazio a d dimensioni e assumiamo che tali dati siano stati generati da una singola distribuzione gaussiana con parametri μ e Σ, che però non conosciamo.

Il nostro obiettivo è stimare questi parametri. Indichiamo quindi con $\theta = (\mu, \Sigma)$ l'insieme dei parametri del modello e cerchiamo il valore più probabile di θ dati i dati osservati:

$$\arg\max_{\theta} P(\theta|X)$$

Applicando il teorema di Bayes, possiamo riscrivere questa quantità come:

$$\arg\max_{\theta} \frac{P(X|\theta)\, P(\theta)}{P(X)}.$$

L'espressione $P(X)$ rappresenta la probabilità di generare i dati ed è indipendente dai parametri θ, quindi può essere omessa senza modificare il risultato dell'ottimizzazione.

Allo stesso modo, possiamo assumere di non avere alcuna informazione a priori sui parametri (ipotesi di prior non informativo), e quindi considerare $P(\theta)$ costante. Di conseguenza, il problema si riduce a massimizzare la quantità $P(X|\theta)$:

$$\hat{\theta} = \arg\max_{\theta} P(X|\theta) = \arg\max_{\theta} P(\theta|X)$$

Questa quantità prende il nome di **verosimiglianza** (likelihood). È importante osservare che la verosimiglianza ha la stessa forma di una funzione di densità di probabilità, ma viene interpretata in modo diverso: in questo caso i dati sono fissati e i parametri sono le variabili.

3.4 MLE per una gaussiana singola

Possiamo interpretare la funzione di densità gaussiana in due modi differenti. Da un lato, possiamo vederla come una funzione dei dati, dati i parametri:

$$p(x; \mu, \Sigma) = \mathcal{N}(\mu, \Sigma) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\Sigma|^{1/2}}\, e^{-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)},$$

dall'altro lato, possiamo considerarla come una funzione dei parametri, dati i dati, cioè come likelihood:

$$L(\mu, \Sigma; x) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\Sigma|^{1/2}}\, e^{-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)}$$

Sia ora $D = \{x_1, \ldots, x_N\}$ un insieme di dati di training i.i.d. in uno spazio d-dimensionale. In questo caso, la probabilità congiunta dei dati è data dal prodotto delle probabilità dei singoli punti:

$$p(D; \mu, \Sigma) = p(\{x_1, \ldots, x_N\}; \mu, \Sigma) = p(x_1; \mu, \Sigma) \cdot \cdots \cdot p(x_N; \mu, \Sigma)$$

ovvero:

$$p(D;\mu,\Sigma) = \prod_{i=1}^{N} p(x_i;\mu,\Sigma) = \prod_{i=1}^{N} \mathcal{N}(x_i;\mu,\Sigma),$$

lo stesso vale per la likelihood:

$$L(\mu,\Sigma;D) = \prod_{i=1}^{N} \mathcal{N}(x_i;\mu,\Sigma).$$

Per stimare i parametri, cerchiamo i valori che massimizzano la likelihood:

$$\mu^*,\Sigma^* = \arg\max_{\mu,\Sigma} L(\mu,\Sigma;D)$$

Questo problema può essere risolto imponendo che le derivate della likelihood rispetto ai parametri siano nulle:

$$\frac{\partial L(\mu,\Sigma;D)}{\partial \mu} = 0, \quad \frac{\partial L(\mu,\Sigma;D)}{\partial \Sigma} = 0.$$

Massimizzare la verosimiglianza. Per massimizzare la verosimiglianza, possiamo cercare l'argmin di $-L$ (poichè $\arg\max f(x) = \arg\min -f(x)$). In particolare per avere una migliore stabilità numerica, possiamo applicare la funzione logaritmo che, essendo una funzione strettamente monotona, restituisce lo stesso arg min (o argmax). Inoltre è comodo utilizzare il logaritmo in quanto trasforma la produttoria in una sommatoria (per le proprietà dei logaritmi). Quindi si ha che

$$\mu^*,\Sigma^* = \arg\max L(\mu,\Sigma;x) = \arg\max \log(L(\mu,\Sigma;x)) = \arg\min[-\log(L(\mu,\Sigma;x))]$$

e quindi

$$\mu^*,\Sigma^* = -\log\left(\prod_{i=1}^{N} L(\mu,\Sigma;x_i)\right) = -\sum_{i=1}^{N} \log L(\mu,\Sigma;x_i).$$

Questi risultati valgono anche per la probabilità

$$L(\mu,\Sigma;X) = \prod_{i=1}^{N} \mathcal{N}(x_i;\mu,\Sigma)$$

infatti possiamo trovare μ,Σ massimizzando la probabilità

$$\mu^*,\Sigma^* = \arg\max L(\mu,\Sigma;x) = \arg\max \log L(\mu,\Sigma;x)$$

utilizzando l'ipotesi di i.i.d. trasformiamo il prodotto in una somma di logaritmi:

$$\arg\max \log\left(\prod_{i=1}^{N} \mathcal{N}(x_i;\mu,\Sigma)\right) = \arg\max \sum_{i=1}^{N} \log \mathcal{N}(x_i;\mu,\Sigma)$$

sostituendo con l'equazione della gaussiana

$$\arg\max \sum_{i=1}^{N} \log\left(\frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}}\, e^{-\frac{1}{2}(x_i-\mu)^\top \Sigma^{-1}(x_i-\mu)}\right) =$$

$$= \arg\max \sum_{i=1}^{N} \left[\log\left(\frac{1}{(2\pi)^{D/2}}\right) + \log\left(\frac{1}{|\Sigma|^{1/2}}\right) + \log\left(e^{-\frac{1}{2}(x_i-\mu)^\top \Sigma^{-1}(x_i-\mu)}\right)\right] =$$

$$= \arg\max \sum_{i=1}^{N} \left[\text{costante} - \frac{1}{2}\log|\Sigma| - \frac{1}{2}(x_i-\mu)^\top \Sigma^{-1}(x_i-\mu)\right].$$

Troviamo ora μ ponendo la derivata parziale uguale a zero:

$$\frac{\partial L(\mu,\Sigma;x)}{\partial \mu} = 0 \quad \iff \quad \frac{\partial}{\partial \mu}\left[\sum_{i=1}^{N}\left(\text{costante} - \frac{1}{2}\log|\Sigma| - \frac{1}{2}(x_i-\mu)^\top \Sigma^{-1}(x_i-\mu)\right)\right] = 0$$

ma questo corrisponde (usando il gradiente) a calcolare

$$\sum_{i=1}^{N} \nabla_\mu \left[-\frac{1}{2}(x_i-\mu)^\top \Sigma^{-1}(x_i-\mu)\right]$$

con un cambio di variabile $\hat{x} = x_i - \mu$ e $A = \Sigma^{-1}$, calcoliamo $\nabla_{\hat{x}}(\hat{x}^\top A\, \hat{x})$:

$$\nabla_{\hat{x}}(\hat{x}^\top A\, \hat{x}) = \underbrace{\left(A + A^\top\right)}_{A=A^\top} \hat{x} = 2A\hat{x}$$

risostituendo $\hat{x} = x_i - \mu$ e $A = \Sigma^{-1}$ e applicando la regola della catena

$$\sum_{i=1}^{N} -\frac{1}{2}[2\Sigma^{-1}(x_i-\mu)] \cdot (-1) = \sum_{i=1}^{N} \Sigma^{-1}(x_i-\mu).$$

A questo punto poniamo il gradiente uguale a zero, fattorizziamo e semplifichiamo

$$\Sigma^{-1}\left(\sum_{i=1}^{N}(x_i-\mu)\right) = \Sigma^{-1}\left(\sum_{i=1}^{N} x_i - N\mu\right) = 0$$

Poichè l'inversa della covarianza non è singolare, possiamo semplificare e ottenere la stima di massima verosimiglianza

$$\Sigma\Sigma^{-1}\sum_{i=1}^{N} x_i = \Sigma\Sigma^{-1} N\mu \quad \iff \quad \sum_{i=1}^{N} x_i = N\mu \quad \iff \quad \mu^* = \frac{1}{N}\sum_{i=1}^{N} x_i.$$

Ripetendo lo stesso ragionamento per Σ otteniamo:

$$\Sigma^* = \frac{1}{N}\sum_{i=1}^{N}(x_i-\mu)(x_i-\mu)^\top.$$

3.5 Responsibilities nei GMM

Consideriamo un modello di mistura gaussiana costituito da K componenti gaussiane. Introduciamo una variabile latente $z \in \{1, ..., K\}$ dove il valore $z = k$ indica che il datapoint x è stato generato dalla k-esima componente gaussiana. Nel contesto dei modelli generativi probabilistici, uno degli obiettivi principali consiste nel determinare, dato un datapoint osservato x, la probabilità che esso appartenga a una specifica componente del mixture model. Tale probabilità prende il nome di responsibility della componente k-esima rispetto all'osservazione x.

3.5.1 Calcolo delle responsibilities

Per determinare la probabilità a posteriori della variabile latente z, applichiamo il teorema di Bayes esplicitando la normalizzazione al denominatore:

$$p(z|x) = \frac{p(x|z)p(z)}{\sum_z p(x|z)p(z)}.$$

Nel caso particolare in cui si voglia valutare la probabilità che il datapoint x appartenga alla componente k-esima, definiamo:

$$\gamma_k = p(z = k|x);$$

pertanto espandendo i termini

$$\gamma_k = \frac{p(x|z = k)p(z = k)}{\sum_{j=1}^{K} p(x|z = j)p(z = j)}.$$

La quantità γ_k rappresenta quindi la probabilità a posteriori che il punto x sia stato generato dalla gaussiana k-esima.

Interpretazione dei termini. Nel numeratore compare come prodotto $p(x|z = k)p(z = k)$ che può essere interpretato come:

- un termine generativo $p(x|z = k)$ che misura quanto la componente k riesca a spiegare il datapoint x;
- un termine di selezione $p(z = k)$ che rappresenta la probabilità prior di scegliere la componente k

Nel caso dei GMM abbiamo che $p(x|z = k) = \mathcal{N}(x; \mu_k, \Sigma_k)$, mentre $p(z = k) = \pi_k$, dove μ_k è il vettore delle medie della k-esima gaussiana, Σ_k è la matrice di covarianza e π_k è il coefficiente di mixing associato alla componente k tale che $\sum \pi_k = 1$. Sostituendo tali espressioni nella formula precedente otteniamo:

$$\gamma_k = \frac{\pi_k \mathcal{N}(x; \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x; \mu_j, \Sigma_j)}.$$

Significato probabilistico delle responsibilities. Le reponsibilities rappresentano una misura probabilistica del contributo che ciascuna componente del mixture model fornisce nella spiegazione del datapoint osservato. A differenza di altri algoritmi di clustering hard, come K-means, nei GMM un datapoint può appartenere simultaneamente a più cluster e l'appartenenza avviene in maniera probabilistica; per questo motivo si parla di soft clustering.

3.5.2 Algoritmo Expectation-Maximization per GMM

L'apprendimento dei parametri $\{\mu_k, \Sigma_k, \pi_k\}_{k=1}^K$ di un GMM viene effettuato tramite l'algoritmo Expectation-Maximization (EM). L'algoritmo alterna iterativamente due fasi: expectation step (E-step) e maximization step (M-step).

E-step Nell'E-step si assumono fissati i parametri correnti del modello: μ_k, Σ_k, π_k, si calcolano le responsibilities per ciascun datapoint. Per ogni osservazione x_n e ogni componente k:

$$\gamma_{nk} = p(z_n = k|x_n) = \frac{\pi_k \mathcal{N}(x_n; \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_n; \mu_j, \Sigma_j)}.$$

In questa fase non vengono modificati i parametri del modello e si stimano solamente le probabilità di appartenenza dei punti ai cluster.

M-step Nel M-step si mantengono fissate le responsibilities calcolate nell'E-step e si aggiornano i parametri del modello massimizzando la log-likelihood attesa. Definiamo

$$N_k = \sum_{n=1}^N \gamma_{nk}$$

che rappresenta il numero effettivo di datapoint assegnati alla componente k. I nuovi parametri vengono aggiornati come segue:

$$\pi_k = \frac{N_k}{N}, \qquad \mu_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} x_n, \qquad \Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (x_n - \mu_k)(x_n - \mu_k)^\top.$$

L'algoritmo EM alterna quindi due operazioni complementari:

- nell'E-step si determina quanto ogni componente gaussiana sia responsabile della generazione dei datapoint osservati;
- nel M-step si aggiornano i parametri delle gaussiane utilizzando tali probabilità di appartenenza.

Il procedimento viene ripetuto iterativamente fino alla convergenza della log-likelihood del modello.

Esempio. Applichiamo l'algoritmo EM per GMM sul dataset $X = \{1, 2, 8, 9\}$. Supponiamo di voler modellare i dati mediante un GMM con $K = 2$ componenti gaussiane.

L'idea intuitiva è che i punti $1, 2$ appartengano a un cluster e in punti $8, 9$ appartengano a un altro cluster; tuttavia il modello non conosce a priori tali assegnazioni, ma le apprende automaticamente. Supponiamo di inizializzare $\mu_1 = 0, \mu_2 = 10$, $\sigma_1^2 = \sigma_2^2 = 1$ e $\pi_1, \pi_2 = 0.5$. Ricordiamo che la densità gaussiana unidimensionale è

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

Nel passo di expectation si calcolano le responsibilities:

$$\gamma_{nk} = p(z_n = k|x_n) = \frac{\pi_k \mathcal{N}(x_n; \mu_k, \sigma_k^2)}{\sum_{j=1}^{N} \pi_j \mathcal{N}(x_n; \mu_j, \sigma_j^2)}.$$

Ad esempio per il punto $x_1 = 1$:

$$\mathcal{N}(x_1 = 1; 0, 1) \sim 0.242, \quad \mathcal{N}(x_1 = 1; 10, 1) \sim 0$$

quindi

$$\gamma_{11} = \frac{0.5 \cdot 0.242}{0.5 \cdot 0.242 + 0} \sim 1, \qquad \gamma_{12} \sim 0.$$

Ripetendo il procedimento per tutti i punti otteniamo:

x_n	γ_{n1}	γ_{n2}
1	1	0
2	1	0
8	0	1
9	0	1

Nel passo di maximization invece si aggiornano i parametri del modello. Definiamo $N_k = \sum_{n=1}^{N} \gamma_{nk}$. Nel nostro caso $N_1 = N_2 = 2$. I coefficienti di mixing diventano:

$$\pi_1 = \frac{2}{4} = 0.5, \qquad \pi_2 = \frac{2}{4} = 0.5.$$

Le nuove medie sono:

$$\mu_1 = \frac{1 \cdot 1 + 1 \cdot 2 + 0 \cdot 8 + 0 \cdot 9}{2} = 1.5, \qquad \mu_1 = \frac{0 \cdot 1 + 0 \cdot 2 + 1 \cdot 8 + 1 \cdot 9}{2} = 8.5$$

e le nuove varianze risultano:

$$\sigma_1^2 = \frac{(1-1.5)^2 + (2-1.5)^2}{2} = 0.25, \qquad \sigma_2^2 = \frac{(8-8.5)^2 + (9-8.5)^2}{2} = 0.25.$$

Quindi dopo un'iterazione dell'algoritmo EM otteniamo:

$$\mu_1 = 1.5, \quad \mu_2 = 8.5, \quad \sigma_1^2 = \sigma_2^2 = 0.25, \qquad \pi_1 = \pi_2 = 0.5.$$

Ripetendo E-step e M-step iterativamente, le gaussiane convergono progressivamente verso una stima ottimale nella distribuzione dei dati.

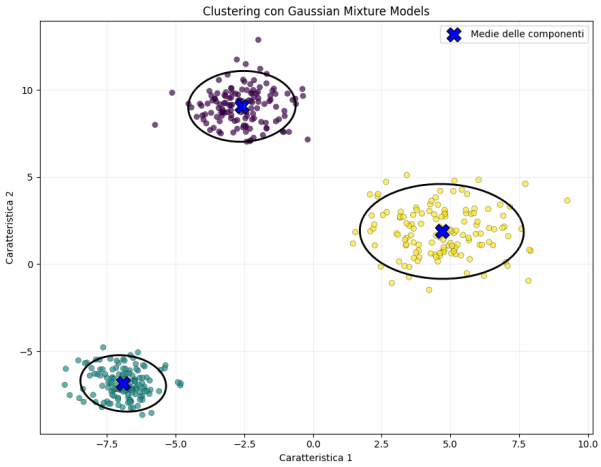


Figura 2: Gaussian mixture models

4 Regressione lineare e polinomiale

4.1 Regressione lineare

Consideriamo un insieme di dati $\{x_i, y_i\}_{i=1}^n$, generati da una distribuzione sconosciuta in cui:

- $x_i \in \mathbb{R}^d$
- $y_i \in \mathbb{R}$.

L'obiettivo della **regressione lineare** è trovare una funzione che, dato un input x, produca un valore y il più vicino possibile a quello osservato. Intuitivamente, dato un insieme di punti non allineati, vogliamo trovare la retta (o, in dimensione più alta, un iperpiano) che meglio li approssima [Figura 2]. Ci poniamo nel caso in cui il numero di dati sia molto maggiore della dimensione: $N \gg d$. Introduciamo quindi una funzione parametrica $f_\theta(x) \approx y$ dove θ rappresenta i parametri del modello. Anche problemi di classificazione possono essere ricondotti a questo schema:

- classificazione binaria: $y \in \{0, 1\}$
- classificazione multiclasse: $y \in \{1, ..., N\}$

in questi casi si passa comunque attraverso una formulazione di tipo regressivo.

Assunzione sui dati. Si assume che i dati siano generati da un modello lineare affetto da rumore gaussiano:

$$y = \theta^* x + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

o equivalentemente possiamo scrivere

$$y \sim \mathcal{N}(\theta^* x, \sigma^2).$$

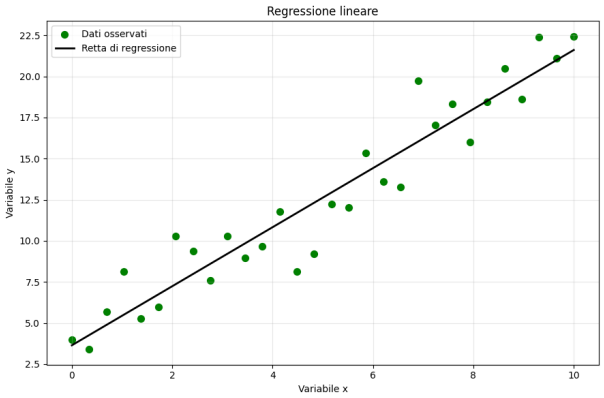


Figura 3: Retta di regressione

Questo significa che:

- se $\sigma = 0$, i dati giacciono perfettamente su una retta
- se $\sigma > 0$, c'è rumore nei dati.

Il parametro σ rappresenta l'errore intrinseco del problema, ovvero il Bayes optimal error. Per stimare i parametri θ esistono diversi approcci equivalenti, fondamentalmente abbiamo:

- metodo analitico (annullando il gradiente)
- metodo probabilistico (MLE)
- metodo geometrico-algebrico (proiezione)
- metodo numerico (gradient descent).

4.1.1 Metodo analitico

La funzione di predizione del modello è $y' = \theta^\top x + \theta_0$. Per semplificare la struttura possiamo incorporare il bias nel vettore delle features estendendo $x \leftarrow [x_i, 1]$ così da poter scrivere semplicemente

$$y' = \theta^\top x.$$

Per misurare quanto la predizione si discosta dal valore reale, utilizziamo l'errore quadratico $(y - y')^2$; estendendo a tutti i dati otteniamo lo scarto quadratico medio:

$$L = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - \theta^\top x_i \right)^2 .$$

Spesso si introduce la funzione di costo

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{N} \left(\theta^\top x_i - y_i\right)^2$$

dove si introduce il fattore $1/2$ per semplificare successivamente il calcolo del gradiente. Ora per lavorare in modo più compatto introduciamo:

- design matrix $X \in \mathbb{R}^{N \times (d+1)}$ che contiene tutti i vettori $x_i^\top$

$$X = \begin{bmatrix} - & x_1^\top & - \\ - & x_2^\top & - \\ & \vdots & \\ - & x_N^\top & - \end{bmatrix}$$

- label matrix $y \in \mathbb{R}^{N \times 1}$ che contiene tutti i valori y_i.

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$$

Le predizioni per tutti i dati si ottengono con l'operazione $X\theta$ dove $\theta \in \mathbb{R}^{(d+1)\times 1}$ è il vettore dei parametri

$$X\theta = \begin{bmatrix} x_1^\top \theta \\ x_2^\top \theta \\ \vdots \\ x_N^\top \theta \end{bmatrix}$$

e di conseguenza il vettore degli errori sarà dato da

$$X\theta - y = \begin{bmatrix} x_1^\top \theta - y_1 \\ x_2^\top \theta - y_2 \\ \vdots \\ x_N^\top \theta - y_N \end{bmatrix}.$$

La funzione totale di loss quadratica totale è la seguente

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{N} \left(x_i^\top \theta - y_i\right)^2 \quad \iff \quad J(\theta) = \frac{1}{2} (X\theta - y)^\top (X\theta - y);$$

consideriamo la forma matriciale ed espandiamo

$$J(\theta) = \frac{1}{2} \left[(X\theta)^\top (X\theta) - 2(X\theta)^\top y + y^\top y \right] = \frac{1}{2} \left[\theta^\top X^\top X\theta - 2\theta^\top X^\top y + y^\top y \right]$$

calcoliamo il gradiente rispetto a θ e lo poniamo uguale a 0 ottenendo l'equazione normale

$$\nabla_\theta J(\theta) = X^\top X\theta - X^\top y = 0 \quad \Longrightarrow \quad X^\top X\theta = X^\top y.$$

Se $X^\top X$ è invertibile allora $\theta = (X^\top X)^{-1}X^\top y$, dove $X^+ = (X^\top X)^{-1}X^\top$ è definita pseudoinversa di X, quindi $\theta = X^+ y$. Possiamo interpretare geometricamente questo problema come:

- $X\theta$ è combinazione lineare delle colonne di X
- La soluzione $\hat{y} = X\theta = X(X^\top X)^{-1}X^\top y$ è la proiezione ortogonale di y sul subspace generato da X.

4.1.2 Metodo probabilistico

Per introdurre una visione probabilistica della regressione lineare, assumiamo che ogni valore y_i sia generato da un modello lineare a cui si somma un rumore gaussiano:

$$y_i = \theta^\top x_i + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2).$$

In questo contesto osserviamo le coppie (x_i, y_i) ma:

- non conosciamo il parametro θ
- non conosciamo il rumore stocastico ε

Lo scarto tra osservazione e modello diventa quindi $\varepsilon = y_i - \theta^\top x_i$, ma per ipotesi $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, questo significa che gli errori sono indipendenti e identicamente distribuiti (i.i.d.) secondo una distribuzione normale. Possiamo quindi scrivere la distribuzione di y_i dato x_i:

$$p(y_i|x_i;\theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \theta^\top x_i)^2}{2\sigma^2}\right)$$

questa quantità coincide anche con la loss vista come funzione dei parametri:

$$p(y_i|x_i;\theta) \doteq L(\theta; x_i, y_i).$$

La distribuzione è centrata in $\theta^\top x_i$, cioè $y_i|x_i;\theta \sim \mathcal{N}(\theta^\top x_i, \sigma^2)$. Se assumiamo che i dati siano indipendenti, la probabilità congiunta (likelihood) è

$$\prod_{i=1}^{n} p(y_i|x_i;\theta)$$

l'obiettivo di MLE è trovare

$$\theta = \arg\max_\theta \prod_{i=1}^{n} p(y_i|x_i;\theta);$$

per semplificare i calcoli, si massimizza la log-likelihood che diventa

$$l(\theta) = n \log \frac{1}{\sqrt{2\pi\sigma^2}} - \frac{1}{2\sigma^2} \sum_{i=1}^{n} \left(y^{(i)} - \theta^\top x^{(i)}\right)^2.$$

Massimizzare la log-likelihood equivale è equivalente a

$$\arg\max_\theta \left[n \log \frac{1}{\sqrt{2\pi\sigma^2}} - \frac{1}{2\sigma^2} \sum_{i=1}^{n} \left(y^{(i)} - \theta^\top x^{(i)}\right)^2 \right]$$

Il primo termine è costante rispetto a θ, quindi il problema si riduce a

$$\arg\min_\theta \frac{1}{2} \sum_{i=1}^{n} \left(y^{(i)} - \theta^\top x^{(i)}\right)^2.$$

Concludiamo che il metodo probabilistico con MLE porta esattamente alla minimizzazione dell'errore quadratico.

4.1.3 Metodo algebrico (proiezione)

Consideriamo il sistema $X\theta \approx y$. Questo sistema in generale non è risolvibile esattamente perchè la matrice X è rettangolare $n \times d$. Ricordiamo che la design matrix X è

$$X = \begin{bmatrix} - & x_1^\top & - \\ - & x_2^\top & - \\ & \vdots & \\ - & x_n^\top & - \end{bmatrix}.$$

Il vettore delle predizioni $\hat{y} = X\theta$ può essere visto come

$$\hat{y} = \theta_0 \ \mathrm{col}_1(X) + \theta_1 \ \mathrm{col}_2(X) + \cdots + \theta_d \ \mathrm{col}_{d+1}(X)$$

cioè una combinazione lineare delle colonne di X. Il caso ideale è quello in cui è possibile scrivere $y = X\theta$ perchè potremmo trovare un iperpiano che interpola perfettamente tutti i punti. In generale questo non è possibile, quindi scegliamo la combinazione lineare che produce un vettore $\hat{y}$ il più vicino possibile a y. Questo equivale a proiettare y sul sottospazio generato dalle colonne di X.

$$\hat{y} = X\theta = X(X^\top X)^{-1} X^\top y, \qquad \theta = (X^\top X)^{-1} X^\top y.$$

L'errore commesso è $\sum_i (y_i - \hat{y}_i)^2$ che in forma compatta diventa

$$\sum_{i=1}^{n} \left(y - X(X^\top X)^{-1} X^\top y\right)^2.$$

Quindi si ha che il training loss è zero solamente quando $y \in$ span delle colonne di X.

4.1.4 Metodo numerico (gradient descent)

Un metodo numerico semplice per risolvere il problema della regressione lineare è il **gradient descent**, in cui i parametri θ vengono aggiornati iterativamente secondo la regola:

$$\theta_{\text{new}} = \theta_{\text{old}} - \varepsilon \nabla_\theta L(\theta; x, y)$$

dove $\nabla_\theta L(\theta; x, y)$ è il gradiente della funzione di loss rispetto ai parametri e $\varepsilon > 0$ è il learning rate. L'idea del metodo è quella di minimizzare la funzione di perdita muovendosi nella direzione opposta a quella del gradiente, cioè nella direzione di massima discesa. La loss viene calcolata su tutto il training set, si inizializzano i parametri θ a zero oppure in modo casuale e si ripete iterativamente l'aggiornamento, osservando la convergenza della sequenza:

$$\{\theta^{(0)}, \theta^{(1)}, \theta^{(2)}, \dots\}$$

La funzione di costo globale è definita come:

$$J(\theta; x, y) = \frac{1}{2} \sum_{i=1}^{n} L(y_i, f_\theta(x_i))$$

da cui il problema di ottimizzazione diventa:

$$\hat{\theta} = \arg\min_\theta \frac{1}{2} \sum_{i=1}^{n} (f_\theta(x_i) - y_i)^2$$

La funzione J è una funzione quadratica convessa e la sua hessiana è:

$$\nabla_\theta^2 J = X^\top X$$

che è una matrice semidefinita positiva. Poiché J è limitata inferiormente e cresce a infinito, essa ammette almeno un minimo. Nel caso in cui:

- $\text{rank}(X) = \min\{d, n\}$ allora $X^\top X$ è definita positiva e la funzione J è strettamente convessa, quindi il minimo è unico.
- $\text{rank}(X) < \min\{d, n\}$ allora J non è strettamente convessa e il minimo non è unico.

Il gradient descent consiste quindi nell'inizializzare i parametri (a zero con il vettore $0^\top$ oppure casualmente campionando i pesi da una distribuzione), ripetere fino a convergenza il calcolo del gradiente della loss sull'intero training set e aggiornare i parametri nella direzione opposta al gradiente:

$$\theta \leftarrow \theta - \gamma \nabla_\theta J(\theta; x, y)$$

dove γ è il learning rate. Nel caso della regressione lineare, il gradiente esplicito è:

$$\nabla_\theta J(\theta) = \sum_{i=1}^{n} (\theta^\top x_i - y_i) x_i$$

quindi l'aggiornamento diventa:

$$\theta \leftarrow \theta - \gamma \sum_{i=1}^{n} (\theta^\top x_i - y_i) x_i$$

La convergenza si considera raggiunta quando:

- la validation loss non migliora più (early stopping)
- la variazione della funzione di costo è trascurabile: $|J(\theta_t) - J(\theta_{t-1})| \approx 0$
- i parametri non cambiano più significativamente: $\|\theta_t - \theta_{t-1}\| \approx 0$
- la norma del gradiente tende a zero: $\|\nabla_\theta J(\theta; x, y)\| \to 0$

Quando una di queste condizioni è soddisfatta, la stima finale dei parametri è $\theta = \hat{\theta}$.

Stochastic gradient descent. Una variante della discesa del gradiente è stochastic gradient descent. Spesso si usa questa variante perché invece di usare **tutto il dataset** ogni volta (come nel Gradient Descent classico), si usa **un campione casuale**, rendendo l'algoritmo più veloce e rumoroso (spesso aiuta a evitare minimi locali). Questo metodo però presenta alcune limitazioni, principalmente oscilla molto e di conseguenza non converge in modo 'pulito', infatti converge in un intorno $\mathcal{B}_\gamma$ della soluzione.

4.2 Regressione polinomiale

Per estendere la regressione lineare al caso non lineare, è possibile adottare diverse strategie. Tra le principali:

- l'utilizzo di **basis functions**, che porta alla regressione polinomiale e ai metodi kernel;
- l'utilizzo di modelli di **Deep Learning**, in cui si considerano trasformazioni del tipo $Wx + b$ seguite da funzioni di attivazione non lineari (ad esempio ReLU).

In questa sezione ci concentriamo sulla regressione polinomiale.

4.2.1 Modello lineare

Sia dato un dataset $D = \{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathbb{R}^d,\ y_i \in \mathbb{R}$. Nel modello lineare si assume:

$$f_\theta(x) = \sum_{i=1}^{d} \theta_i x_i + \theta_0.$$

Introducendo esplicitamente il termine di bias ponendo $x_0 = 1$, possiamo scrivere:

$$x = [1, x_1, \ldots, x_d] \in \mathbb{R}^{d+1}, \quad \theta = [\theta_0, \theta_1, \ldots, \theta_d],$$

da cui:

$$f_\theta(x) = \sum_{i=0}^{d} \theta_i x_i = \theta^\top x.$$

Questo modello è lineare e non è in grado di rappresentare relazioni non lineari tra x e y.

4.2.2 Estensione tramite basis functions

Per introdurre non linearità, si considera una trasformazione:

$$\phi : \mathbb{R}^d \to \mathbb{R}^m, \quad \phi(x) = [1, \phi_1(x), \ldots, \phi_m(x)].$$

Il modello diventa:

$$f_\theta(x) = \sum_{i=0}^{m} \theta_i \phi_i(x) = \theta^\top \phi(x).$$

Il modello rimane lineare nei parametri θ, ma non lineare rispetto alle variabili di input.

4.2.3 Regressione polinomiale

Un caso importante è dato da basis functions polinomiali:

$$\phi_i(x) = x^i.$$

Si ottiene così un modello polinomiale di grado m (tipicamente $m > d$).

- Le funzioni polinomiali sono **globali**: una variazione locale dei dati influisce sull'intero dominio.
- Questo può portare a instabilità e scarsa capacità di generalizzazione.
- Una possibile soluzione consiste nell'utilizzo di **spline**, che suddividono lo spazio degli input in regioni locali.

4.2.4 Overfitting

Si consideri un modello generativo:

$$y = f(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2).$$

All'aumentare della complessità del modello (ad esempio aumentando il grado del polinomio), si può incorrere in **overfitting**, in cui il modello:

- interpola perfettamente i dati di training;
- non cattura la struttura sottostante.

Un indicatore tipico è la crescita eccessiva dei coefficienti θ.

4.2.5 Regolarizzazione L2

Per controllare la complessità del modello si introduce:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{n} \left(\theta^\top \phi(x_i) - y_i\right)^2 + \frac{\lambda}{2} \|\theta\|_2^2,$$

dove $\lambda > 0$ è un iperparametro.

- Il primo termine rappresenta l'errore di fitting;
- il secondo termine è la **regolarizzazione** (L2 o weight decay).

L'obiettivo è:

$$\hat{\theta} = \arg\min_{\theta} J(\theta).$$

4.2.6 Soluzione analitica e interpretazione spettrale

Indicando con X la matrice dei dati trasformati, si ottiene:

$$(X^\top X + \lambda I)\theta = X^\top y,$$

da cui:

$$\hat{\theta} = (X^\top X + \lambda I)^{-1} X^\top y.$$

La matrice $X^\top X + \lambda I$ è definita positiva e quindi invertibile. Sia:

$$X^\top X = U \Sigma U^\top, \quad \Sigma = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_d^2).$$

$$X^\top X = U\Sigma U^\top = U \begin{bmatrix} \sigma_1^2 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_d^2 \end{bmatrix} U^\top \qquad \Longrightarrow \qquad X^\top X + \lambda \mathbb{I} = U \begin{bmatrix} \sigma_1^2 + \lambda & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_d^2 + \lambda \end{bmatrix} U^\top$$

e invertendo la matrice

$$(X^\top X + \lambda \mathbb{I})^{-1} = \begin{bmatrix} \frac{1}{\sigma_1^2+\lambda} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \frac{1}{\sigma_d^2+\lambda} \end{bmatrix}$$

La regolarizzazione aumenta gli autovalori piccoli, riducendo la varianza dello stimatore. Ora analizziamo il bias

$$y = X\hat{\theta} + \varepsilon,$$

si ottiene:

$$\hat{\theta}_n = (X^\top X + \lambda I)^{-1} X^\top X \hat{\theta} + (X^\top X + \lambda I)^{-1} X^\top \varepsilon.$$

Facendo l'aspettativa ed esplicitando otteniamo

$$\mathbb{E}[\hat{\theta}] = [(X^\top X + \lambda \mathbb{I})^{-1} X^\top X]\hat{\theta}$$

$$= U \begin{bmatrix} \frac{1}{\sigma_1^2+\lambda} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \frac{1}{\sigma_d^2+\lambda} \end{bmatrix} U^\top X^\top X \hat{\theta} = U \begin{bmatrix} \frac{1}{\sigma_1^2+\lambda} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \frac{1}{\sigma_d^2+\lambda} \end{bmatrix} \underbrace{U^\top U}_{=\mathbb{I}} \begin{bmatrix} \sigma_1^2 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_d^2 \end{bmatrix} U^\top \hat{\theta}$$

$$= \begin{bmatrix} \frac{1}{\sigma_1^2+\lambda} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \frac{1}{\sigma_d^2+\lambda} \end{bmatrix} \begin{bmatrix} \sigma_1^2 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_d^2 \end{bmatrix} U^\top \hat{\theta} = \begin{bmatrix} \frac{\sigma_1^2}{\sigma_1^2+\lambda} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \frac{\sigma_d^2}{\sigma_d^2+\lambda} \end{bmatrix} U^\top \hat{\theta}.$$

Si può allora concludere che

- Se $\lambda = 0$, allora $\mathbb{E}[\hat{\theta}_n] = \hat{\theta}$ (stimatore non distorto).
- Se $\lambda > 0$, si introduce un **bias**, ma si riduce la varianza.

Si ottiene quindi il classico **trade-off bias-varianza**.

5 Classificazione

5.1 Perceptron e logistic regression

Abbiamo visto che il modello lineare è della forma $\hat{y} = \theta^\top x$, dove θ rappresenta il vettore dei valori da stimare. Nel caso della linear regression si può interpretare l'output come $\hat{y} = \sigma(\theta^\top x)$ dove la funzione $\sigma(y)$ è la funzione indentità, cioè $\sigma(y) = y$. In diversi modelli di regressione e classificazione si utilizzano invece funzioni di attivazione differenti. Ad esempio, nel perceptron si utilizza spesso la funzione segno:

$$\sigma(z) = \text{sgn}(z)$$

mentre nella logistic regression si adotta la funzione sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

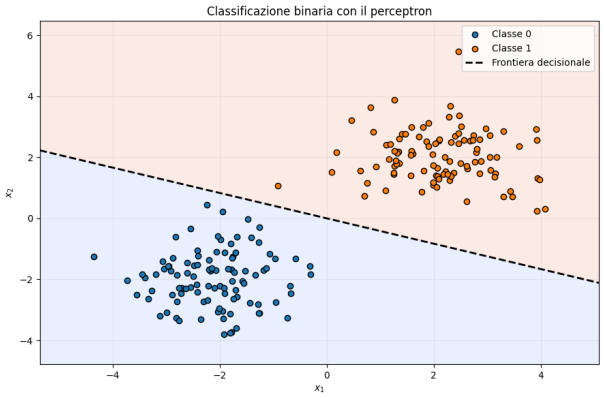


Figura 4: Esempio di classificazione binaria del perceptron

5.1.1 Algoritmo del perceptron

Consideriamo dati $x_i \in \mathbb{R}^d$ ed etichette $y \in \{0, 1\}$, e defininiamo il modello:

$$f_\theta(x) = \sigma(\theta^\top x),$$

dove

$$\sigma(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Questa scelta non è particolarmente efficace, in quanto la funzione non è differenziabile in $z = 0$ ed è costante a tratti. Inoltre, la magnitudine (modulo) del prodotto scalare $\theta^\top x$ non è rilevante: ciò che conta è unicamente il segno. In particolare vale che:

- $\theta^\top x > 0$ quando l'angolo tra θ e x è minore di $\pi/2$
- $\theta^\top x = 0$ quando l'angolo tra θ e x è esattamente $\pi/2$
- $\theta^\top x < 0$ quando l'angolo tra θ e x è maggiore di $\pi/2$.

Nel perceptron, per classificare il punto, il parametro θ viene aggiornato secondo la regola:

$$\theta \leftarrow \theta - \varepsilon(\theta^\top x - y)x$$

dove il learning rate ε viene scelto opportunamente. In alternativa, si può descrivere l'aggiornamento in termini geometrici; se un punto è classificato erroneamente possiamo scegliere di aggiornare θ al variare dell'angolo tra θ e x come segue:

$$\begin{cases} \theta \leftarrow \theta - \gamma x & \text{se l'angolo è minore di } \pi/2 \\ \theta \leftarrow \theta + \gamma x & \text{se l'angolo è maggiore di } \pi/2 \end{cases}$$

i punti classificati correttamente invece vengono mantenuti invariati. Vale inoltre la relazione:

$$\theta^\top x \leq (\theta + \gamma x)^\top x = \theta^\top x + \underbrace{\gamma x^\top x}_{\text{positivo}}.$$

L'inizializzazione può essere effettuata ponendo tutti i parametri uguali a zero, cioè $\theta = 0^\top$, oppure inizializzandoli casualmente secondo una distribuzione scelta. Il procedimento viene iterato fino al raggiungimento della convergenza:

$$\theta \leftarrow \theta - \gamma(\sigma(\theta^\top x) - y)x \quad \text{oppure} \quad \theta \leftarrow \theta + \gamma(\sigma(\theta^\top x) - y)x.$$

Quando la convergenza viene raggiunta, la stima finale dei parametri è data da θ. Nel caso corretto (sia per punti positivi che negativi), θ rimane invariato. In caso contrario, θ diventa una combinazione lineare dei campioni di training $\{x_i, y_i\}$. Dato un insieme di training linearmente separabile, il perceptron trova una delle infinite soluzioni che classificano correttamente tutti i punti. In questo caso, la convergenza è garantita, anche se la soluzione trovata non è necessariamente ottima.

Tuttavia, il percettrone presenta alcuni svantaggi: il ciclo continua all'infinito se i dati non sono linearmente separabili o comunque anche se fossero separabili potrebbe essere necessario un elevatissimo tempo di esecuzione.

Algoritmo perceptron.
$w(0)$ scelto in modo casuale
for $k = 0, 1, ...$
  scegliere $(x^p, y^p) \in TS$
  se $y^p w(0)^\top x^p < 0$
    $w(k+1) = w(k) + y^p x^p$
  altrimenti
    $w(k+1) = w(k)$
  end for

Apprendimento di XOR. Il perceptron non è in grado di apprendere la funzione logica XOR (grafico in [Figura 5]). Si potrebbe pensare di sostituire $\phi^\top x$ con $\theta^\top \phi(x)$, introducendo una trasformazione non lineare, ad esempio nel contesto della regressione polinomiale. Ad esempio un problema bidimensionale può essere risolto proiettandolo in uno spazio a quattro dimensioni con l'aggiunta di un bias.

Esempio. Supponiamo di voler classificare una point cloud composta da due insiemi di punti disposti in due cerchi concentrici, uno interno all'altro, quindi non linearmente separabili nello spazio originale. Per affrontare questo problema con il perceptron, possiamo introdurre una trasformazione non lineare delle feature. Se il vettore originale è $x = [x_1, x_2]$, si considera una mappatura nello spazio delle features estese:

$$x' = [x_1, x_2, x_1^2 + x_2^2].$$

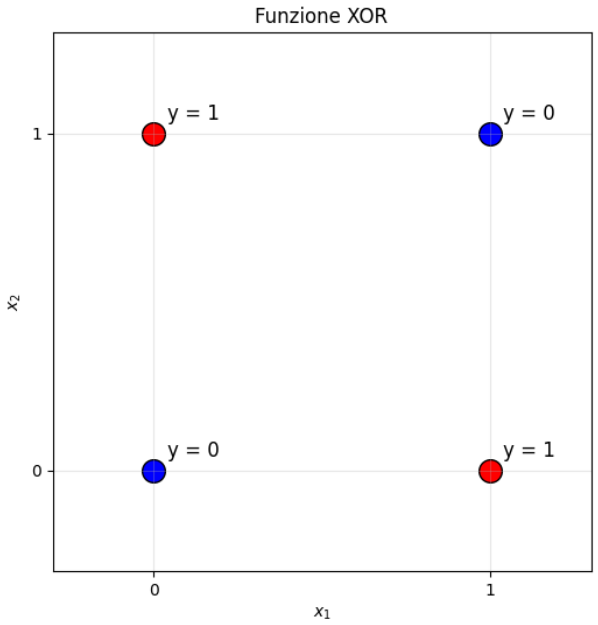


Figura 5: Grafico della funzione XOR

Osserviamo che il termine $x_1^2 + x_2^2$ corrisponde a $\|x\|_2^2$ (alla norma al quadrato del vettore x). Questa trasformazione introduce una variabile che dipende esclusivamente dalla distanza del punto dall'origine. In questo nuovo spazio, i punti appartenenti al cerchio esterno avranno valori di $\|x\|_2^2$ più grandi, mentre i punti appartenenti al cerchio più piccolo avranno valori di $\|x\|_2^2$ più piccoli. In questo modo, la separazione tra le due classi diventa linearmente realizzabile nello spazio trasformato, e un classificatore lineare come il perceptron può essere applicato efficacemente in quanto consideriamo solamente la distanza dall'origine.

5.1.2 Logistic regression

La logistic regression è un modello discriminativo utilizzato come classificatore binario; si tratta di un algoritmo molto diffuso e frequentemente impiegato. Le etichette sono binarie, come nel caso precedente, e il modello è definito come:

$$f_\theta(x) = \sigma(\theta^\top x)$$

dove viene impiegata la funzione logistica (o anche chiamata sigmoide) definita da

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

che soddisfa le seguenti proprietà: $\sigma(z) \to 0$ per $z \to -\infty$ e $\sigma(z) \to 1$ per $z \to +\infty$.

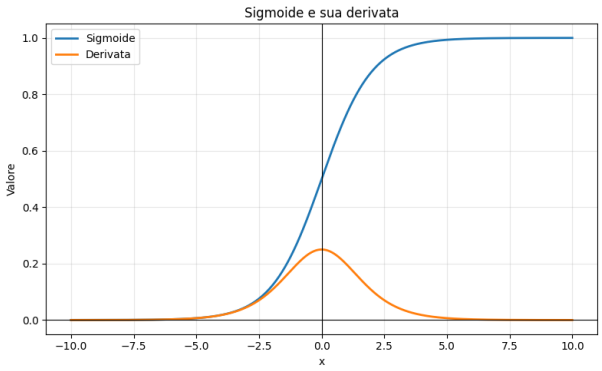


Figura 6: Grafico della funzione logistica e la sua derivata prima

Possiamo quindi eseguire la classificazione modellando la probabilità condizionata $p(y|x)$ come:

$$\begin{cases} p(y=1|x;\theta) = f_\theta(x) \\ p(y=0|x;\theta) = 1 - f_\theta(x) \end{cases}$$

da cui segue direttamente

$$f_\theta(x) = \frac{1}{1+e^{-\theta^\top x}}.$$

I punti che si trovano sul *decision boundary* sono quelli tali che $\theta^\top x = 0$, ossia i punti per i quali la funzione assume valore $1/2$. In generale .5 è il threshold (valore soglia) utilizzato per la classificazione.

Possiamo riscrivere il modello in forma probabilistica compatta tramite una distribuzione di Bernoulli:

$$p(y|x;\theta) = (f_\theta)^y(1-f_\theta)^{1-y}.$$

In questo modo, invece di assumere rumore gaussiano come nella regressione lineare, si assume che il rumore segua una distribuzione di Bernoulli.

Modelli lineari generalizzati (GLM). Per la predizione dei dati è possibile utilizzare diverse tecniche a seconda della natura della variabile target:

- $y \in \mathbb{R}$: distribuzione gaussiana o laplaciana, si utilizza regressione;
- $y \in \{0,1\}$: distribuzione di Bernoulli, si utilizza classificazione binaria;
- $y \in \{1,...,K\}$: distribuzione categorica, si utilizza classificazione multiclasse;
- $y \in \mathbb{N}_+$: distribuzione di Poisson, si utilizza Poisson regression.

Quindi in generale si sceglie una distribuzione in base alla natura dei dati (support compatibile con il tipo di variabile osservata) e si esprime la distribuzione scelta tramite la famiglia esponenziale di funzioni, modellando f_θ e stimando i parametri θ.

5.1.3 Apprendimento dei parametri.

Per apprendere i parametri della logistic regression si massimizza, come di consueto, la log-likelihood:

$$L(\theta; x, y) = \prod_{i=1}^{n} p\left(y^{(i)}|x^{(i)}; \theta\right) = \prod_{i=1}^{n} \left(f_\theta(x^{(i)})\right)^{y^{(i)}} \left(1 - f_\theta(x^{(i)})\right)^{1-y^{(i)}}.$$

Applicando il logaritmo si ottiene:

$$l(\theta) = \sum_{i=1}^{n} y^{(i)} \log f_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - f_\theta(x^{(i)})).$$

Questo corrisponde alla cosiddetta log-loss.

Ora prima di calcolare il gradiente $\nabla_\theta \log L(\theta)$, per semplificare i passaggi successivi, consideriamo la derivata della funzione sigmoide:

$$\sigma'(z) = \frac{-1}{(1+e^{-z})^2}(-e^{-z}) = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}}.$$

Questa espressione può essere riscritta come:

$$\sigma'(z) = \frac{1}{1+e^{-z}}\left(1 - \frac{1}{1+e^{-z}}\right) \quad \iff \quad \sigma'(z) = \sigma(z)(1-\sigma(z)).$$

Si ottiene quindi che il gradiente è

$$\sum_{i=1}^{n} \left[y^{(i)} - \sigma(\theta^\top x^{(i)})\right] x^{(i)}.$$

Tutti i principali problemi supervisionati parametrici trattati sono convessi rispetto ai parametri. La loss, in funzione dei parametri, è convessa. In particolare lo sono: least squares, logistic regression, support vector machines, multi-class logistic e tutti i modelli con regolarizzazione $L2$. La discesa del gradiente, come già visto, consente di trovare un minimo (o massimo) globale nel caso di funzioni convesse, rendendo semplice il problema di ottimizzazione.

5.2 Multiclass classification

Occupiamoci ora della classificazione multi-classe. Anche in questo caso il problema può essere ricondotto a una formulazione binaria, ma invece di considerare etichette scalari $y \in \{0, 1\}$, si modellano etichette vettoriali $\vec{y} \in \{0, 1\}^K$ dove K è il numero di classi. In particolare, il vettore y è un vettore *one-hot*: vale $y_i = 1$ per la classe corretto (indice detto *ground truth*) e $y_j = 0$ per tutte le altre classi $j \neq i$.

5.2.1 Divergenza tra distribuzioni

Consideriamo due distribuzioni $p(x)$ e $q(x)$. L'obiettivo è stimare una misura di distanza (più precisamente, una divergenza) tra di esse. Supponiamo che $p(x)$ rappresenti la distribuzione reale delle etichette e $q(x)$ quella stimata dal modello.

Se utilizziamo $q(x)$ per costruire uno schema di codifica al posto di $p(x)$, l'informazione media aggiuntiva necessaria per rappresentare x è data da:

$$H(P,Q) - H(P) = -\sum_{x\in X} p(x)\log q(x) + \sum_{x\in X} p(x)\log p(x).$$

Questa quantità è detta divergenza di Kullback-Leibler, e può essere riscritta come:

$$KL(p\|q) = \sum_{x\in X} p(x)\log\frac{p(x)}{q(x)}.$$

in particolare vale che la divergenza è nulla se e solo se le distribuzioni coincidono, formalmente $KL(p\|q) = 0 \iff p = q$. La divergenza $KL(\cdot\|\cdot)$ non è commutativa. Per renderla simmetrica si può introdurre la Jensen-Shannon divergence:

$$JS = \frac{1}{2}KL(p\|m) + \frac{1}{2}KL(q\|m)$$

dove $m = (p+q)/2$.

Nota sulla cross-entropy. Nel caso della classificazione, l'entropia delle etichette è nulla (nel caso one-hot), quindi rimane solamente il termine

$$H(P,Q) = -\sum_{x\in X} p(x)\log q(x)$$

che prende il nome di cross-entropy.

5.2.2 Score functions

Supponiamo ora di avere una variabile categorica $y \in \{1,...,K\}$. Se i dati sono separabili, possiamo usare funzioni lineari della forma $w^\top x + b > 0$, dove w è il vettore normale all'iperpiano e b il bias. Nel caso multiclasse, si considerano più iperpiani, uno per classe. Se lo spazio è suddiviso da più frontiere decisionali, un punto può trovarsi tra diverse regioni: in tal caso si assegna la classe in base alla distanza maggiore dall'iperpiano associato (cioè al valore più alto dello score). Analogamente, se il punto si trova in una regione 'negativa' rispetto a tutte le classi, si assegna comunque la classe corrispondente al valore $w^\top x + b$ meno negativo (cioè il massimo tra i valori).

5.2.3 Passaggio dal caso binario a multiclasse

Nel caso binario abbiamo visto che la classificazione è data da $\text{sgn}(w^\top x + b)$ che assegna $+1$ o -1 in base al segno. Nel caso multiclasse si introduce un insieme di parametri: $\{w_k, b_k\}_{k=1}^K$, uno per ciascuna classe. Un punto $\hat{x}$ viene assegnato alla classe $\hat{k}$ se:

$$f_{w,b}(\hat{x}) > f_{w,j}(\hat{x}) \quad \forall j \neq \hat{k}.$$

In forma esplicita quindi abbiamo che:

$$\begin{aligned} f_{w_1,b_1} &= w_1^\top x + b_1, \quad y = 1 \\ f_{w_2,b_2} &= w_2^\top x + b_2, \quad y = 2 \\ &\vdots \\ f_{w_K,b_K} &= w_K^\top x + b_K, \quad y = K \end{aligned}$$

Tutto questo può essere scritto in forma matriciale come

$$Z = WX + b.$$

5.2.4 Ottimizzazzione e softmax

Dato che in classificazione multi-classe il modello che abbiamo definito seleziona la classe con score più alto, vorremmo ottimizzare la funzione

$$\nabla_{W,b} \arg\max_k (WX + b).$$

Tuttavia, la funzione arg max non è differenziabile e, analogamente alla funzione segno, ha derivata nulla quasi ovunque, fornendo quindi poca informazione per l'ottimizzazione.

Per questo motivo si introduce la softmax, una funzione differenziabile che sostituisce l'argmax. La softmax trasforma un vettore reale (logits) in una distribuzione di probabilità:

$$\sigma_i(z) = \frac{\exp z_i}{\sum_{k=1}^K \exp z_i}.$$

Possiamo interpretare $z = Wx+b$ come una log-probabilità non normalizzata per ciascuna classe:

$$p(y|x; W, b) = \frac{\exp(W_y x + b_y)}{\sum_{k=1}^K \exp(W_k x + b_k)}.$$

Quindi la cross-entropy sul dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ è

$$L(\mathcal{D}, W, b) = \frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} -\log\left(\frac{\exp s_y}{\sum_{k=1}^K \exp s_k}\right)$$

dove s_y è lo score della classe corretta y ground truth e s_k lo score della classe k.

Osservazione importante. Supponiamo di inizializzare il modello con pesi W, b casuali non nulli, e che i dati x siano informativi (non nulli). In tal caso, il classificatore multiclasse si comporta inizialmente come un classificatore casuale. Di conseguenza la loss attesa è approssimativamente $\sim \log K$.

Relazione tra softmax e sigmoide. Nel caso $K = 2$, la softmax si riduce alla funzione logistica. Definendo $z = Wx + b$, si ha:

$$p(y = 1|x; W, b) = \frac{\exp z_1}{\exp z_1 + \exp z_2} \cdot \frac{\exp(-z_1)}{\exp(-z_1)} = \frac{1}{1 + \exp[-(z_1 - z_2)]}.$$

Quindi si ottiene una sigmoide applicata a $z_1 - z_2$. In questo modo non è necessario modellare esplicitamente $p(y = 0|x)$, poichè vale

$$p(y = 0|x) = 1 - p(y = 1|x).$$

6 Reti neurali

Consideriamo la discesa del gradiente della funzione di loss rispetto ai parametri θ usando l'intero training set composto da n esempi. Questa variante prende il nome di **Batch Gradient Descent** (Batch-GD):

$$\theta \leftarrow \theta - \gamma \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta J(\theta; x_i, y_i),$$

dove γ è il learning rate e $J(\theta)$ la loss associata al singolo campione. Nel caso in cui il gradiente venga calcolato utilizzando un singolo esempio scelto casualmente dal training set, si ottiene lo **Stochastic Gradient Descent** (SGD):

$$\theta \leftarrow \theta - \gamma \nabla_\theta J(\theta; x_i, y_i).$$

6.1 Mini-batch training

Per l'addestramento delle reti neurali, che tipicamente presentano un elevato numero di parametri e una funzione obiettivo non convessa, si utilizza una strategia intermedia nota come mini-batch gradient descent.

In tale approccio, ad ogni iterazione vengono campionati k esempi dal training set, con $k \ll n$, spesso scelto come potenza di due per ragioni computazionali. L'aggiornamento dei parametri assume quindi la forma:

$$\theta \leftarrow \theta - \gamma \frac{1}{k} \sum_{i=1}^{k} \nabla_\theta J(\theta; x_i, y_i).$$

In pratica il training avviene come segue:

- si effettua uno *shuffle* del training set X
- si suddivide il dataset in mini-batch disgiunti di dimensione k
- si eseguono aggiornamenti sequenziali sui singoli mini-batch.

Questo procedimento corrisponde a un campionamento uniforme senza reinserimento. Una volta che viene svolto su tutti i dati, prende il nome di **epoch**. L'addestramento di una rete neurale può quindi essere descritto sia in termini di numero di iterazioni, sia in termini di numero di epoche.

Esempio: rappresentazione dei dati. Le immagini RGB di dimensione $H \times W$ possono essere rappresentate come tensori a tre canali. Un mini-batch contenente k immagini ha dimensione:

$$H \times W \times 3 \times k,$$

dove H è l'altezza, W la larghezza, 3 corrisponde ai canali RGB e k la dimensione del mini-batch. Nel caso dei video, la rappresentazione si estende ulteriormente includendo la dimensione temporale:

$$H \times W \times 3 \times t \times k,$$

dove t indica il numero di frame temporali.

6.1.1 SGD su mini-batch

L'addestramento di reti neurali richiede generalmente un'inizializzazione casuale dei parametri, soprattutto nel caso di funzioni obiettivo non strettamente convesse. Tipicamente si assume $\theta \sim \mathcal{N}(\cdot)$, evitando inizializzazioni nulle che potrebbero compromettere la simmetria del modello. Ad ogni iterazione si calcola il gradiente della loss sul mini-batch corrente $\nabla_\theta L(\theta)$, e si aggiornano i parametri nella direzione opposta al gradiente medio:

$$\theta \leftarrow \theta - \gamma \frac{1}{k} \sum_{i=1}^{k} \nabla_\theta J(\theta; x_i, y_i).$$

Il procedimento viene ripetuto iterativamente fino al raggiungimento della convergenza o di un criterio di arresto. Il mini-batch può essere interpretato come un compromesso tra:

- Batch gradient descent, che utilizza l'intero dataset;
- Stochastic gradient descent, che utilizza un singolo esempio.

La presenza di stocasticità introduce rumore nel gradiente, il quale può risultare utile nell'ottimizzazione di funzioni non convesse, favorendo l'esplorazione dello spazio dei parametri e riducendo la probabilità di convergere a minimi locali subottimali.

6.2 Momentum

Una tecnica fondamentale per stabilizzare l'ottimizzazione è il momentum, che introduce una memoria delle direzioni di aggiornamento precedenti. Si considera inizialmente una media mobile:

$$\mu_t = \frac{t-1}{t}\mu_{t-1} + \frac{1}{t}x_{new},$$

che può essere generalizzata mediante una media mobile esponenziale:

$$\mu_{t+1} = \alpha\mu_t + (1-\alpha)x_{new}, \qquad 0 \leq \alpha \leq 1.$$

6.2.1 SGD con momentum

Nel contesto dello stochastic gradient descent, il momentum viene implementato introducendo una variabile ausiliaria che accumula le direzioni di aggiornamento passate:

- tale variabile agisce come una memoria esponenzialmente pesata dei gradienti,
- riduce le oscillazioni lungo direzioni ad alta varianza,
- accelera la convergenza nelle direzioni consistenti.

Formalmente:

$$\Delta_t = \alpha\Delta_{t-1} + (1-\alpha)\frac{1}{k}\sum_{i=1}^{k}\nabla_\theta J(\theta; x_i, y_i),$$

con $\alpha \approx .9$ nella pratica. L'aggiornamento dei parametri è quindi:

$$\theta \leftarrow \theta - \gamma\Delta_t.$$

L'inizializzazione dei parametri riveste un ruolo cruciale, in particolare nei modelli non convessi come le reti neurali. Per evitare simmetrie e stagnazioni nell'apprendimento, i parametri vengono inizializzati casualmente $\theta \sim \mathcal{N}(\cdot)$, evitando inizializzazione nulle o degeneri. Successivamente, il processo di ottimizzazione viene iterato fino a convergenza o fino all'applicazione di criteri di arresto anticipato (early stopping), producendo la stima finale dei parametri del modello.

6.3 Ottimizzazione convessa e non convessa

6.3.1 Loss surface in linear regression

Consideriamo la funzione di loss l_2^2 nella regressione lineare con un vettore di parametri $\theta \in \mathbb{R}^2$. In questo caso è possibile visualizzare geometricamente la loss surface, cioè la superficie della funzione obiettivo rispetto ai parametri del modello. La loss assume una forma convessa, tipicamente assimilabile a un paraboloide:

- esiste un unico minimo globale,

- l'ottimizzazione tramite gradient descent risulta relativamente semplice.

Aumentando la dimensionalità dei dati x e quindi anche il numero di parametri θ, non è più possibile rappresentare graficamente la loss surface. Tuttavia, la funzione di loss rimane comunque convessa, e quindi continua a esistere un unico minimo globale.

6.3.2 Deep Learning e ottimizzazione non convessa

Nel caso delle reti neurali profonde, il problema di ottimizzazione cambia radicalmente. La funzione obiettivo diventa infatti non covessa: compaiono numerosi minimi locali, saddle points, regioni piatte, vallate strette (valleys), colline (hills) e superfici noisy. Inoltre il numero di parametri può diventare estremamente elevato, rendendo l'ottimizzazione molto più complessa rispetto ai modelli lineari classici.

6.4 Struttura di una rete neurale

Il multi-layer perceptron (MLP) è una rete neurale completamente connessa. Per comprendere la sua struttura, consideriamo inizialmente il caso di una rete lineare a singolo strato, equivalente a una regressione softmax lineare. L'ultimo layer di classificazione di una rete neurale può essere rappresentato come la seguente pipeline:

$$x \quad \Longrightarrow \quad z = Wx + b \quad \Longrightarrow \quad e^z \quad \Longrightarrow \quad p = \frac{e^z}{\sum_k e^z} \quad \Longrightarrow \quad -\log p_y.$$

In questo contesto z rappresenta il vettore dei logits, la softmax trasforma i logits in probabilità e $-\log p_y$ corrispondente alla cross-entropy loss relativa alla classe corretta.

6.4.1 Rappresentazione di un singolo layer

Consideriamo il regressore softmax lineare: $z = Wx + b$. Tale operazione può essere interpretata come un layer lineare seguito da una funzione di attivazione non lineare: $\sigma(Wx + b)$. Esplicitamente:

$$\sigma(Wx+b) = \sigma \circ \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1d} \\ w_{21} & w_{22} & \cdots & w_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ w_{k1} & w_{k2} & \cdots & w_{kd} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_k \end{bmatrix} = \sigma \circ \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_k \end{bmatrix}.$$

Il termine $w_k^\top x + b_k$ rappresenta l'attivazione associata alla k-esima classe, mentre il vettore riga w_k rappresenta un neurone (o unità) del layer. La non linearità viene introdotta dalla funzione di attivazione:

$$\sigma(w_k^\top x + b_k).$$

Limiti di un singolo layer lineare. Una rete costituita da un unico layer lineare seguito da softmax può risultare insufficiente per modellare problemi altamente non lineari. Per aumentare la capacità espressiva del modello, si introduce un ulteriore layer non lineare prima del classificatore finale. L'idea consiste nell'apprendere una trasformazione: $x \to \phi(x)$, che mappa i dati originali in uno spazio delle feature più espressivo. In questo modo il classificatore non opera direttamente sugli input x, ma sulle feature apprese $\phi(x)$.

6.4.2 Rete neurale a due layer

Indichiamo con l'apice il numero del layer e con il pedice l'indice dell'unità. Consideriamo:

$$W^1 \in \mathbb{R}^{d \times p}, \quad b^1 \in \mathbb{R}^p,$$

e

$$W^2 \in \mathbb{R}^{p \times k}, \quad b^2 \in \mathbb{R}^k.$$

La rete quindi assume la forma:

$$p = \sigma\left(W^2\left(\sigma(W^1 x + b^1)\right) + b^2\right).$$

Dal punto di vista dimensionale:

$$d \to p \to k,$$

dove d è la dimensione di ingresso dell'input, p il numero di neuroni "nascosti" e k il numero di classi. Le funzioni di attivazione vengono applicate element-wise.

6.5 Funzione di attivazione ReLU

La funzione Rectified Linear Unit (ReLU) è una funzione $\sigma(z)$ che vale 0 per $z < 0$ e z per $z > 0$. Scritta in modo compatto è definita come:

$$\sigma(z) = \max(0, z)$$

ReLu è una funzione piece-wise lineare.

6.5.1 Confronto tra ReLU e funzione logistica

La funzione logistica, introdotta in precedenza, viene spesso utilizzata per modellare probabilità, ma raramente viene impiegata nei layer intermedi delle reti profonde. Tra gli svantaggi principali:

- richiede il calcolo dell'esponenziale
- può causare il problema dei vanishing gradients per valori grandi dell'input.

La ReLU invece presenta diversi vantaggi:

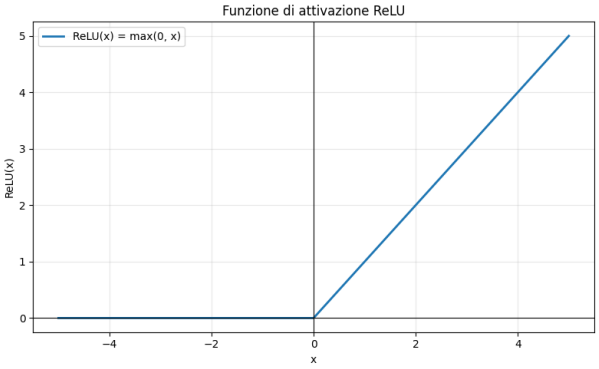


Figura 7: Grafico della funzione ReLU

- è computazionalmente più efficiente
- riduce il problema dei vanishing gradients
- converge significativamente più velocemente della sigmoide
- è semplice da implementare

Tuttavia non propaga gradiente per valori negativi e non è differenziabile nell'origine. Per questo motivo esistono numerose varianti per le funzioni di attivazioni da utilizzare in base alla casistica del problema, come ad esempio tanH, Leaky ReLU, Parametric ReLU ed ELU. L'aggiunta di layer non lineari aumenta quindi la capacità espressiva della rete neurale.

Osservazione sul numero di parametri addestrabili. Consideriamo un MLP con input in $\mathbb{R}^d$, un primo layer con p neuroni e un layer finale con k neuroni corrispondenti alle classi. Assumendo la presenza del bias in tutti i layer:

- il primo layer richiede $d \times p + p$ parametri,
- il secondo layer richiede $p \times k + k$ parametri.

Il numero totale di parametri addestrati è quindi:

$$(d \times p + p) + (p \times k + k).$$

6.6 Inizializzazione di una rete neurale

L'inizializzazione dei parametri rappresenta un aspetto fondamentale nell'addestramento delle reti neurali profonde. Una scelta inadeguata dei pesi iniziali può infatti compromettere la propagazione dell'informazione attraverso la rete e rendere inefficiente, o addirittura impossibile, il processo di ottimizzazione.

Sia data una rete neurale composta da più layer. Per il layer l-esimo indichiamo:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = \sigma\left(z^{(l)}\right),$$

dove $W^{(l)}$ è la matrice dei pesi, $b^{(l)}$ è il vettore dei bias, $a^{(l-1)}$ rappresenta l'attivazione del layer precedente, σ è la funzione di attivazione applicata elemento per elemento. Nel caso del primo layer si assume $a^{(0)} = x$.

6.6.1 Inizializzazione nulla

Supponiamo di inizializzare tutti i pesi e bias a zero, ovvero:

$$W^{(l)} = 0, \quad b^{(l)} = 0 \qquad \forall l.$$

Considerando, ad esempio la funzione sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

si ottiene immediatamente che

$$z^{(l)} = 0 \quad \Longrightarrow \quad a^{(l)} = \sigma(0) = .5.$$

Di conseguenza si ha che:

- tutti i neuroni di uno stesso layer producono la medesima attivazione,
- l'output della rete risulta indipendente dall'input,
- la rete si comporta come una funzione costante.

Questo problema non riguarda solamente il forward pass, ma soprattutto il backward pass. Durante il calcolo del gradiente, infatti, tutti i neuroni di uno stesso layer ricevono esattamente lo stesso aggiornamento:

$$\frac{\partial L}{\partial W_{ij}^{(l)}} = \frac{\partial L}{\partial W_{kj}^{(l)}} \qquad \forall i, k.$$

Dunque tutti i neuroni apprendono le stesse feature, la rete perde capacità rappresentativa e il training diventa inefficace. Tale fenomeno prende il nome di **problema della simmetria**.

6.6.2 Inizializzazione costante non nulla

Potrebbe sembrare naturale inizializzare i pesi con una costante non nulla:

$$W_{ij}^{(l)} = c, \quad c \neq 0.$$

Tuttavia anche questa scelta mantiene la simmetria tra i neuroni. Infatti:

- ogni neurone applica la stessa trasformazione lineare,
- le attivazioni risultano copie scalate tra loro,
- i gradienti rimangono identici.

Formalmente, i vettori colonna della matrice dei pesi restano linearmente dipendenti durante tutto l'addestramento. Ne consegue che i neuroni di uno stesso layer rimangono indistinguibili, la rete utilizza di fatto una sola feature per layer e l'espressività del modello si riduce drasticamente.

6.6.3 Inizializzazione casuale asimmetrica

Per rompere la simmetria è necessario inizializzare i parametri in modo casuale: $W_{ij}^{(l)} \sim \mathcal{D}$, dove $\mathcal{D}$ è una distribuzione opportuna. Ad esempio possiamo scegliere $\mathcal{D} \sim \mathcal{U}$ oppure $\mathcal{D} \sim \mathcal{N}$ e avere valori scalari nella matrice W campionati come:

$$W_{ij} \sim \mathcal{U}(-.1, .1) \quad \text{oppure} \quad W_{ij} \sim \mathcal{N}(0, .01).$$

Grazie alla casualità i neuroni partono da configurazioni differenti, i gradenti diventano diversi e ciascun neurone apprende features differenti. L'inizializzazione casuale rappresenta quindi il meccanismo che consente alla rete neurale di sviluppare rappresentazioni interne ricche e non ridondanti.

6.6.4 Xavier/He initialization

Nelle reti profonde, una semplice inizializzazione casuale può comunque risultare insufficiente. Durante il forward pass e il backward pass, infatti, le attivazioni e i gradienti possono esplodere o annullarsi progressivamente.

Per mitigare tali problemi vengono utilizzate inizializzazioni progettate per preservare la varianza dei segnali lungo la rete. Un esempio è la **Xavier Initialization** che può essere espressa tramite distribuzione gaussiana o uniforme; rispettivamente:

$$W_{ij}^{(l)} \sim \mathcal{N}\left(0, \frac{2}{n^{(l)} + n^{(l-1)}}\right) \quad \text{oppure} \quad W_{ij}^{(l)} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n^{(l)} - n^{(l-1)}}}, \sqrt{\frac{6}{n^{(l)} - n^{(l-1)}}}\right)$$

Qui $n^{(l-1)}$ è il numero di neuroni del layer precedente, e $n^{(l)}$ è il numero di neuroni del layer corrente. Inoltre con questa scelta si vuole cercare di mantenere $\text{Var}\left(a^{(l)}\right) \approx \text{Var}\left(a^{(l-1)}\right)$, evitando che il segnale cresca o decada eccessivamente attraversando i layer.

Nel caso di funzioni ReLU, la Xavier Initialization non è ottimale, poiché metà delle attivazioni vengono annullate. La **He Initialization** modifica quindi la varianza:

$$W_{ij}^{(l)} \sim \mathcal{N}\left(0, \frac{2}{n^{(l-1)}}\right).$$

Questa scelta compensa l'effetto della ReLU e migliora significativamente la propagazione del gradiente nelle reti profonde.

6.7 Reti neurali profonde

Una rete neurale profonda può essere interpretata come una composizione gerarchica di trasformazioni non lineari:

$$y = f_L \circ f_{L-1} \circ \cdots \circ f_1(x).$$

Ciascun layer apprende una rappresentazione delle feature costruita sopra quella del layer precedente: i layer iniziali apprendono feature semplici, i layer intermedi apprendono strutture più complesse e i layer finali apprendono rappresentazioni ad alto livello utili per il task finale. Dal punto di vista funzionale:

$$y = f \circ (\sigma \circ f) \circ \cdots \circ (\sigma \circ f)(x).$$

6.7.1 Teorema di approssimazione universale

Uno dei risultati teorici fondamentali sulle reti neurali è il Teorema di approssimazione universale. Sia $f : \mathbb{R}^d \to \mathbb{R}^k$ una funzione continua definita su un compatto. Allora $\forall \varepsilon > 0$ esiste una rete neurale feed-forward con un singono hidden layer, un numero finito di neuroni, una funzione di attivazione non lineare tale che

$$|f(x) - NN_\theta(x)| \leq \varepsilon, \quad \forall x.$$

Il teorema garantisce dunque la capacità teorica delle reti neurali di approssimare arbitrariamente bene funzioni continue. Tuttavia:

- non fornisce una procedura costruttiva per determinare i parametri ottimali,
- non specifica il numero necessario di neuroni,
- non affronta i problemi computazionali legati all'ottimizzazione.

6.8 Backpropagation

L'addestramento di una rete neurale richiede il calcolo del gradiente della loss rispetto a tutti i parametri del modello:

$$\nabla_\theta L.$$

Poiché una rete moderna può contenere milioni di parametri, il calcolo manuale delle derivate risulta impraticabile.

6.8.1 Differenze finite

Una possibile approssimazione del gradiente deriva dalla definizione numerica di derivata:

$$\frac{df}{dx}(x) = \lim_{\varepsilon \to 0} \frac{f(x+\varepsilon) - f(x)}{\varepsilon}.$$

Dato un parametro $w = W_{ij}$ si procede valutando la loss corrente $L(w)$, perturbando il parametro $w \leftarrow w+\varepsilon$ e ricalcolando la loss $L(w+\varepsilon)$. Infine si può stimare numericamente la derivata, ponendo ad esempio $\varepsilon \sim 10^{-5}$, con

$$\frac{\partial L}{\partial w} \approx \frac{L(w+\varepsilon) - L(w)}{\varepsilon}.$$

Ripetendo il procedimento per tutti i parametri si ottiene una stima numerica del gradiente. Tuttavia il costo computazionale cresce linearmente con il numero di parametri e il metodo risulta troppo lento per l'addestramento, infatti viene principalmente usato per il gradient checking.

6.8.2 Descrizione di backpropagation

Il metodo standard per il calcolo efficiente dei gradienti è il **backpropagation**, basato sull'applicazione iterata della regola della catena.

Una rete neurale può essere rappresentata come un **grafo computazionale aciclico diretto** (*Directed Acyclic Graph*, DAG):

- i nodi rappresentano operazioni matematiche,
- gli archi rappresentano il flusso dei dati.

L'algoritmo si divide in due fasi:

1. **forward pass**: calcolo delle attivazioni e della loss;
2. **backward pass**: propagazione dei gradienti della loss verso i parametri.

6.8.3 Autodiff e autograd

I moderni framework di deep learning, come PyTorch, implementano automaticamente il backpropagation tramite sistemi di automatic differentiation (autodiff) e autograd. Consideriamo

$$f(x) = x^\top Ax + b^\top x + c.$$

La funzione può essere interpretata come funzione dell'input x o come funzione dei parametri $\theta = (A, b, c)$. Nel contesto dell'apprendimento automatico il dataset viene considerato fissato e l'obiettivo consiste nell'ottimizzare i parametri. Framework come PyTorch costruiscono dinamicamente il grafo computazionale durante il forward pass e calcolano automaticamente i gradienti durante il backward pass.

7 Clustering

7.1 Definizione del problema

Il problema del *clustering* consiste nel suddividere un insieme di vettori in un numero prefissato di gruppi, detti *cluster*, in modo da raggruppare tra loro gli elementi caratterizzati da una maggiore somiglianza. La formulazione matematica del problema dipende dalla misura di dissimilarità adottata e conduce a differenti approcci algoritmici.

Sia $d : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}_+$ una misura di dissimilarità tra due vettori di $\mathbb{R}^n$. Per due vettori $x, y \in \mathbb{R}^n$, un esempio di tale misura è la distanza euclidea:

$$d(x, y) = \|x - y\|.$$

L'obiettivo è individuare una partizione dei dati che minimizzi la dissimilarità complessiva all'interno dei cluster. A tal fine, si definisce la dissimilarità di un cluster come la somma delle dissimilarità tra ciascun elemento assegnato al cluster e il relativo centroide. Sia dato un insieme di N vettori, detto *training set*,

$$TS = \{x^i \in X \subseteq \mathbb{R}^n,\ i = 1, \dots, N\},$$

e sia M il numero prefissato di cluster. Introduciamo le variabili binarie di assegnazione

$$\delta_{ij} = \begin{cases} 1 & \text{se } x^i \text{ è assegnato al cluster } j, \\ 0 & \text{altrimenti,} \end{cases}$$

e i vettori continui $z_j \in \mathbb{R}^n$, che rappresentano i centroidi dei rispettivi cluster.

La dissimilarità del cluster j è definita come

$$s_j = \sum_{i=1}^{N} \delta_{ij}\, d(x^i, z_j).$$

Poiché ogni vettore deve appartenere a uno e un solo cluster, le variabili di assegnazione devono soddisfare i vincoli

$$\sum_{j=1}^{M} \delta_{ij} = 1, \qquad i = 1, \dots, N.$$

Il problema di clustering può quindi essere formulato come il seguente problema di ottimizzazione:

$$\begin{aligned} \min_{\delta, z} \quad & \sum_{i=1}^{N} \sum_{j=1}^{M} \delta_{ij}\, d(x^i, z_j) \\ \text{s.t.} \quad & \sum_{j=1}^{M} \delta_{ij} = 1, && i = 1, \dots, N, \\ & \delta_{ij} \in \{0, 1\}, && i = 1, \dots, N,\ j = 1, \dots, M, \\ & z_j \in \mathbb{R}^n, && j = 1, \dots, M. \end{aligned}$$

Una formulazione alternativa si ottiene rilassando il vincolo di integralità delle variabili di assegnazione e imponendo soltanto la loro non negatività:

$$\begin{aligned}
\min_{\delta,z} \quad & \sum_{i=1}^{N}\sum_{j=1}^{M} \delta_{ij}\, d(x^i, z_j) \\
\text{s.t.} \quad & \sum_{j=1}^{M} \delta_{ij} = 1, && i = 1,\dots,N, \\
& \delta_{ij} \geq 0, && i = 1,\dots,N,\ j = 1,\dots,M, \\
& z_j \in \mathbb{R}^n, && j = 1,\dots,M.
\end{aligned}$$

In questa seconda formulazione, le variabili di assegnazione possono assumere valori continui. Per un centroide fissato, tuttavia, esiste sempre una soluzione ottima con assegnazioni binarie, poiché il problema relativo a ciascun vettore è lineare e ammette una soluzione ottima in un vertice del simplesso. Per semplicità, faremo riferimento alla formulazione rilassata, adottando la seguente notazione compatta:

$$\begin{aligned}
\min_{\delta,z} \quad & f(\delta, z) = \sum_{i=1}^{N} \delta_i^\top d_i(z) \\
\text{s.t.} \quad & e^\top \delta_i = 1, && i = 1,\dots,N, \\
& \delta_i \in \mathbb{R}^M, \quad \delta_i \geq 0, && i = 1,\dots,N, \\
& z_j \in \mathbb{R}^n, && j = 1,\dots,M,
\end{aligned}$$

dove $e \in \mathbb{R}^M$ è il vettore avente tutte le componenti uguali a 1 e

$$d_i(z) = \begin{bmatrix} d(x^i, z_1) \\ d(x^i, z_2) \\ \vdots \\ d(x^i, z_M) \end{bmatrix}.$$

Ottimizzazione rispetto alle variabili di assegnazione. Fissato il vettore dei centroidi $\bar{z}$, il problema di ottimizzazione rispetto a δ si decompone in N sottoproblemi indipendenti, uno per ciascun vettore del training set. L'i-esimo sottoproblema è

$$\begin{aligned}
\min_{\delta_i} \quad & f_i(\delta_i, \bar{z}) = d_i(\bar{z})^\top \delta_i \\
\text{s.t.} \quad & e^\top \delta_i = 1, \\
& \delta_i \geq 0.
\end{aligned}$$

Si tratta di un problema di programmazione lineare in cui le variabili appartengono al simplesso standard. Una soluzione ottima si ottiene assegnando il vettore x^i al cluster il cui centroide presenta la dissimilarità minima. Indicando con

$$j^* \in \arg\min_{j=1,\dots,M} d(x^i, \bar{z}_j),$$

uno degli indici che realizza il minimo, si pone

$$\delta_{ij}^* = \begin{cases} 1 & \text{se } j = j^*, \\ 0 & \text{altrimenti.} \end{cases}$$

Nel caso in cui più centroidi realizzino lo stesso valore minimo, è possibile scegliere uno qualsiasi degli indici corrispondenti.

7.2 Algoritmo k-means

Una delle tecniche di clustering più diffuse è l'algoritmo k-means, che utilizza la distanza euclidea per misurare la dissimilarità tra i dati e i centroidi. In particolare, si considera la funzione

$$d(x, z) = \frac{1}{2}\|x - z\|^2.$$

Il problema di clustering assume quindi la forma

$$\begin{aligned} \min_{\delta, z} \quad & f(\delta, z) = \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{M}\delta_{ij}\|x^i - z_j\|^2 & & \\ \text{s.t.} \quad & \sum_{j=1}^{M}\delta_{ij} = 1, & & i = 1, \ldots, N, \\ & \delta_{ij} \geq 0, & & i = 1, \ldots, N,\ j = 1, \ldots, M, \\ & z_j \in \mathbb{R}^n, & & j = 1, \ldots, M. \end{aligned}$$

L'algoritmo k-means può essere interpretato come un metodo di ottimizzazione a blocchi, nel quale si alternano la minimizzazione rispetto alle variabili di assegnazione δ e quella rispetto ai centroidi z. Nella versione *batch*, a ogni iterazione vengono aggiornate tutte le assegnazioni e, successivamente, tutti i centroidi.

1. Aggiornamento delle assegnazioni. Fissati i centroidi $\bar{z}$, il problema rispetto a δ si decompone in N sottoproblemi indipendenti, ciascuno risolvibile analiticamente. Ogni vettore viene assegnato al cluster il cui centroide è più vicino:

$$j^*(i) \in \arg\min_{j=1,\ldots,M} \|x^i - \bar{z}_j\|.$$

Le nuove variabili di assegnazione sono pertanto

$$\delta_{ij}^* = \begin{cases} 1 & \text{se } j = j^*(i), \\ 0 & \text{altrimenti.} \end{cases}$$

2. Aggiornamento dei centroidi. Fissate le variabili di assegnazione $\bar{\delta}$, il problema rispetto a z si decompone in M sottoproblemi quadratici convessi, uno per ciascun cluster. Il sottoproblema relativo al cluster j è

$$\min_{z_j \in \mathbb{R}^n} f_j(z_j) = \frac{1}{2} \sum_{i=1}^{N} \bar{\delta}_{ij} \|x^i - z_j\|^2.$$

La funzione obiettivo è differenziabile e convessa. Il gradiente rispetto a z_j è

$$\nabla_{z_j} f_j(z_j) = \sum_{i=1}^{N} \bar{\delta}_{ij}(z_j - x^i).$$

Imponendo la condizione di stazionarietà

$$\nabla_{z_j} f_j(z_j) = 0,$$

si ottiene

$$\left(\sum_{i=1}^{N} \bar{\delta}_{ij} \right) z_j = \sum_{i=1}^{N} \bar{\delta}_{ij} x^i.$$

Se il cluster contiene almeno un elemento, la soluzione ottima è quindi

$$z_j^* = \frac{\displaystyle\sum_{i=1}^{N} \bar{\delta}_{ij} x^i}{\displaystyle\sum_{i=1}^{N} \bar{\delta}_{ij}}.$$

Nel caso di assegnazioni binarie, questa espressione coincide con la media aritmetica dei vettori appartenenti al cluster. Se invece il cluster è vuoto, il denominatore è nullo e il sottoproblema non determina un centroide univoco: in tal caso, si può mantenere il centroide precedente oppure adottare una specifica strategia di reinizializzazione.

Algoritmo k-means batch

Siano dati i vettori $x^1, \ldots, x^N \in \mathbb{R}^n$ e il numero di cluster M. Si scelga un'inizializzazione z^0 dei centroidi e si ponga $k = 0$.

1. **Aggiornamento delle assegnazioni.**

   Per ogni $i = 1, \ldots, N$, si calcoli

   $$j_k(i) \in \arg\min_{j=1,\ldots,M} \|x^i - z_j^k\|.$$

   Si definiscano quindi le nuove assegnazioni:

   $$\delta_{ij}^{k+1} = \begin{cases} 1 & \text{se } j = j_k(i), \\ 0 & \text{altrimenti.} \end{cases}$$

2. **Aggiornamento dei centroidi.**

   Per ogni $j = 1, \ldots, M$, se il cluster j è non vuoto, si ponga

$$z_j^{k+1} = \frac{\sum_{i=1}^{N} \delta_{ij}^{k+1} x^i}{\sum_{i=1}^{N} \delta_{ij}^{k+1}}.$$

   Se il cluster è vuoto, si mantenga il centroide precedente oppure lo si reinizializzi secondo una strategia prestabilita.

3. **Verifica del criterio di arresto.**

   Se il criterio di arresto è soddisfatto, l'algoritmo termina. Altrimenti, si ponga $k \leftarrow k + 1$ e si ripetano i passi precedenti.

Criterio di arresto. Un possibile criterio di arresto si basa sulla variazione relativa della funzione obiettivo. Fissata una tolleranza $\varepsilon > 0$, si arresta l'algoritmo quando

$$\frac{\left| f(\delta^{k+1}, z^{k+1}) - f(\delta^k, z^k) \right|}{1 + f(\delta^k, z^k)} \leq \varepsilon.$$

In alternativa, è possibile arrestare l'algoritmo quando le assegnazioni non cambiano più tra due iterazioni consecutive.

Proprietà di convergenza. A ogni iterazione, l'aggiornamento delle assegnazioni minimizza la funzione obiettivo rispetto a δ, mantenendo fissi i centroidi. Analogamente, l'aggiornamento dei centroidi minimizza la funzione obiettivo rispetto a z, mantenendo fisse le assegnazioni. Di conseguenza, il valore della funzione obiettivo non aumenta:

$$f(\delta^{k+1}, z^k) \leq f(\delta^k, z^k),$$

$$f(\delta^{k+1}, z^{k+1}) \leq f(\delta^{k+1}, z^k).$$

Poiché la funzione obiettivo è non negativa, la successione dei suoi valori converge.

Inoltre, quando si utilizzano assegnazioni binarie, esistono soltanto un numero finito di possibili configurazioni di δ. Se, in presenza di parità tra centroidi, si adotta una regola di assegnazione deterministica e si gestiscono coerentemente gli eventuali cluster vuoti, l'algoritmo termina in un numero finito di iterazioni, una volta raggiunta una configurazione stabile. La soluzione finale (δ^*, z^*) soddisfa le condizioni di ottimalità rispetto a ciascun blocco di variabili:

$$f(\delta^*, z^*) \leq f(\delta, z^*) \qquad \forall\, \delta \text{ ammissibile},$$

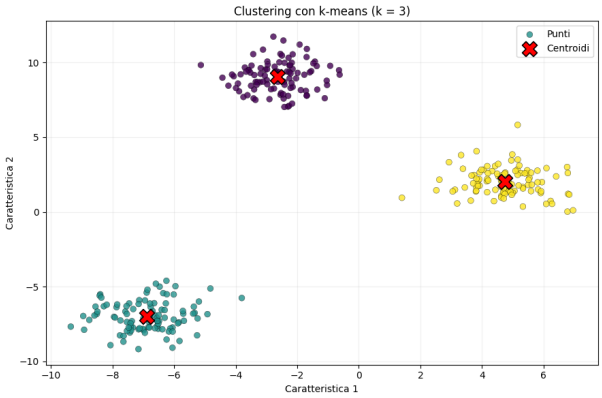


Figura 8: Clustering k-means

e

$$f(\delta^*, z^*) \leq f(\delta^*, z) \qquad \forall\, z \in (\mathbb{R}^n)^M.$$

Queste proprietà esprimono l'ottimalità rispetto a ciascun blocco di variabili preso singolarmente. Non garantiscono, tuttavia, che la soluzione ottenuta sia un minimo globale del problema congiunto, che è in generale non convesso.

8 Valutazione di un modello

8.1 Bias-Variance trade-off

Uno degli obiettivi fondamentali del ML consiste nell'ottenere modelli capaci di generalizzare correttamente su dati non osservati. La qualità della generalizzazione viene misurata tramite il cosiddetto errore di generalizzazione (*generalization error* o *test error*), definito come l'errore commesso dal modello su esempi non utilizzati durante l'addestramento. Dato un dataset composto da N campioni, l'errore empirico medio può essere espresso come:

$$\varepsilon = \frac{1}{N} \sum_{n=1}^{N} L\left(y_n, f(x_n)\right),$$

dove $f(x_n)$ rappresenta la predizione del modello, y_n l'etichetta corretta e $L(\cdot, \cdot)$ una funzione di loss. L'errore di generalizzazione può essere decomposto in tre contributi:

$$\text{errore totale} = \text{bias}^2 + \text{varianza} + \text{errore irriducibile}.$$

8.1.1 Bias

Il **bias** misura l'errore introdotto dalle ipotesi assunte dal modello. Un modello con elevato bias risulta troppo rigido e incapace di rappresentare adeguatamente la reale relazione tra input e output. In tali situazioni il modello non apprende le strutture presenti nei dati

e produce errori elevati sia sul training set che sul test set. Questo fenomeno prende il nome di **underfitting**. In generale modelli troppo semplici hanno un bias elevato.

8.1.2 Varianza

La varianza misura invece la sensibilità del modello rispetto alle variazioni del training set. Un modello con elevata varianza apprende eccessivamente i dettagli specifici dei dati di addestramento, si adatta al rumore presente in essi e generalizza male su nuovi campioni. Questo fenomeno prende il nome di **overfitting**. In generali modelli complessi hanno elevata varianza; tuttavia si tenta di ridurla con tecniche di regolarizzazione.

8.1.3 Decomposizione bias-varianza

Supponiamo che i dati siano generati secondo il modello $y = f(x) + \varepsilon$, dove $\mathbb{E}[\varepsilon] = 0$ e $\mathbb{V}(\varepsilon) = \tau^2$. Sia $\hat{f}_n$ il modello appreso a partire da n camponi di training. Consieriamo il mean squared error su un nuovo campione $\bar{x}$:

$$MSE(\hat{f}_n) = \mathbb{E}\left[(y - \hat{f}_n(\bar{x}))^2\right].$$

Sostituendo $y = f(\bar{x}) + \varepsilon$ si ottiene

$$MSE(\hat{f}_n) = \mathbb{E}\left[[\varepsilon + f(\bar{x}) - \hat{f}_n(\bar{x})]^2\right].$$

Sviluppando il quadrato risulta

$$= \mathbb{E}\left[\varepsilon^2\right] + \mathbb{E}\left[(f(\bar{x}) - \hat{f}_n(\bar{x}))^2\right] + 2\mathbb{E}\left[\varepsilon\right]\,\mathbb{E}\left[f(\bar{x}) - \hat{f}_n(\bar{x})\right].$$

poichè il termine $\mathbb{E}\left[\varepsilon\right] = 0$ il doppio prodotto si annulla e rimane

$$= \mathbb{E}\left[\varepsilon^2\right] + \mathbb{E}\left[(f(\bar{x}) - \hat{f}_n(\bar{x}))^2\right].$$

Sfruttando la relazione $\mathbb{V}(X) = \mathbb{E}\left[X^2\right] - \mathbb{E}\left[X\right]^2$, segue che

$$MSE(\hat{f}_n) = \underbrace{\tau^2}_{\text{errore irriducibile}} + \underbrace{\left(\mathbb{E}\left[\hat{f}_n(\bar{x}) - f(\bar{x})\right]\right)^2}_{\text{errore sistematico}} + \underbrace{\mathbb{V}[\hat{f}_n(\bar{x})]}_{\text{sensibilità ai dati}}.$$

L'errore irriducibile dipende esclusivamente dal rumore presente nel processo generativo e non può essere eliminato da alcun algoritmo di apprendimento.

8.2 Training set, validation set e test set

L'obiettivo dell'apprendimento automatico non consiste nell'ottenere prestazioni ottimali sui dati di training, bensì nel minimizzare l'errore di generalizzazione. Per questo motivo il dataset viene generalmente suddiviso in tre sottoinsiemi:

- training set,
- validation set,
- test set.

Le suddivisioni più comuni (in percentuale) sono 60/20/20, 70/20/10 o 80/10/10.

Training set. Il training set viene utilizzato per apprendere i parametri del modello θ. Durante questa fase si minimizza la funzione di loss. Le prestazioni sul training set tendono ad essere ottimistiche, poichè il modello è ottimizzato direttamente su tali dati.

Validation set. Il validation set viene impiegato per selezionare gli iperparametri, confrontare modelli differenti e controllare fenomeni di overfitting. La scelta finale del modello avviene sulla base delle prestazioni ottenute sul validation set.

Test set. Il test set deve essere utilizzato esclusivamente al termine dell'intero processo di apprendimento. Esso fornisce una stima imparziale dell'errore di generalizzazione del modello finale. È fondamentale che il test set non influenzi: il training, la selezione degli iperparametri e il model selection.

8.3 Cross-validation

Una singola suddivisione training/validation può introdurre elevata variabilità nei risultati, poiché le prestazioni dipendono fortemente dalla particolare scelta casuale dei campioni. Per ridurre questo problema si utilizza la **cross-validation**. Nella cross validation si assume che i dati siano i.i.d., in particolare tutti i campioni devono essere generati dallo stesso processo probabilistico e il processo generativo non deve avere memoria dei campioni precedenti. Analizziamo in particolare la K-fold cross-validation.

Nella K-fold cross-validation il dataset viene suddiviso in K blocchi (folds) di dimensione simile. Ad ogni iterazione:

- un fold viene utilizzato come validation set,
- i rimanenti $K-1$ fold costituiscono il training set.

Il procedimento viene ripetuto K volte, utilizzando ogni fold esattamente una volta come validation set. La prestazione finale viene ottenuta mediando i risultati:

$$\text{score} = \frac{1}{K}\sum_{i=1}^{K}\text{score}_i.$$

La cross validation presenta diversi vantaggi come la riduzione dello spreco di dati, la diminuzione della dipendenza rispetto ad uno specifico split e la produzione di stime più robuste dell'errore di generalizzazione; tuttavia richiede alti costi computazionali (ad esempio se si usa 10-fold cross-validation e si hanno 5 combinazioni di iperparametri, sarà necesario addestrare $10 \times 5 = 50$ modelli differenti).

Scelta di K. I valori di K più comuni sono $2, 5, 10$ (nella pratica 10 è la scelta più diffusa). Valori elevati di K riducono il bias della stima ma aumentano il costo computazionale.

8.3.1 LOO cross-validation

Quando $K = N$, la K-fold cross-validation prende il nome di Leave-One-Out cross-validation (LOO). In questo caso ogni iterazione utilizza un singolo campione come validation set e tutti gli altri campioni vengono usati per il training (quindi si può usare anche con dataset molto piccoli). Tuttavia, questo metodo ha elevati costi computazionali e induce alta variabilità nella stima.

8.4 Regolare gli iperparametri

Gli iperparametri sono parametri che non vengono appresi automaticamente durante in training. Esempi tipici sono il numero di vicini in k-NN, profondità di un decision tree, numero di layer di una rete neurale o numero di centroidi in K-means. La ricerca degli iperparametri richiede: un modello, uno spazio dei parametri, una strategia di ricerca, uno schema di cross-validation e una metrica di valutazione.

Grid search. La grid search esplora esaustivamente tutte le combinazioni possibili degli iperparametri all'interno di griglie predefinite; è semplice da implementare ma computazionalmente costosa.

Random search. La random search campiona casualmente configurazioni di iperparametri. Rispetto alla grid search risulta più efficiente e consente di controllare direttamente il budget computazionale.

Successive halving. L'algoritmo di successive halving realizza una selezione iterativa delle configurazioni candidate. L'idea è la seguente:

1. tutte le configurazioni vengono inizialmente valutate con poche risorse;
2. solo le migliori vengono mantenute,
3. le risorse allocate aumentano progressivamente nelle iterazioni successive.

Le risorse possono corrispondere a numero di campioni, numero di epoche, numero di alberi in una random forest o altri parametri computazionali.

8.5 Metriche di valutazione per la classificazione

Nella maggior parte dei problemi di machine learning, la funzione di costo utilizzata durante l'addestramento rappresenta soltanto un'approssimazione (surrogate objective) del reale obiettivo applicativo. Le metriche di valutazione permettono quindi di tradurre un obiettivo pratico in una quantità numerica misurabile. Le metriche:

- consentono di confrontare modelli differenti,
- guidano il processo di ottimizzazione,
- permettono di diagnosticare problemi di bias o varianza,
- quantificano il divario tra prestazioni desiderate e prestazioni ottenute.

Idealmente, la funzione obiettivo ottimizzata durante il training dovrebbe coincidere con la metrica finale di valutazione; tuttavia ciò non è sempre possibile. Per questo motivo le metriche svolgono un ruolo fondamentale nella valutazione dei modelli.

8.5.1 Classificazione binaria e multiclasse

Consideriamo un problema di classificazione binaria, dove x rappresenta l'input, $y \in \{0, 1\}$ rappresenta l'etichetta e il modello produce una predizione $\hat{y} = h(x)$. I classificatori possono essere suddivisi in due categorie principali:

1. modelli che restituiscono direttamente una classe discreta (k-NN, decision tree);
2. modelli che restituiscono una probabilità nell'intervallo $[0, 1]$ (logistic regression, SVM probabilistiche).

8.5.2 Accuracy

La metrica più semplice è l'**accuracy**, definita come la frazione di campioni classificati correttamente. Dato un dataset di n campioni:

$$\text{accuracy}(y, \hat{y}) = \frac{1}{n} \sum_{i=0}^{n-1} \mathbf{1}(\hat{y}_i = y_i),$$

dove $\mathbf{1}()$ è la funzione indicatrice che vale 1 quando la condizione è vera e 0 altrimenti. L'accuracy assume valori compresi tra 0 e 1.

8.5.3 Top-K Accuracy

In problemi multiclasse con molte categorie, può essere utile rilassare il criterio di correttezza introducendo la Top-K accuracy. Una predizione è considerata corretta se la classe reale compare tra le prime K classi con probabilità maggiore. Formalmente:

$$\text{top-k-accuracy}\left(y, \hat{f}\right) = \frac{1}{n}\sum_{i=0}^{n-1}\sum_{j=1}^{K}\mathbf{1}(\hat{f}_{ij} = y_i).$$

Questa metrica è ampiamente utilizzata in image classification, recommendation systems e large-scale recognition tasks.

8.5.4 Confusion matrix

Per analizzare in modo più dettagliato le prestazioni di un classificatore binario si utilizza la confusion matrix. Essa distingue quattro casi possibili:

Classe reale	**Predizione positiva**	**Predizione negativa**
Positiva	True positive, tp	False negative, fn
Negativa	False positive, fp	True negative, tn

dove tp sono i campioni positivi classificati correttamente, tn i campioni negativi classificati correttamente, fp i campioni negativi classificati erroneamente come positivi e fn i campioni positivi classificati erroneamente come negativi. I termini fp e fn sono anche chiamati errore di tipo I e errore di tipo II. L'accuracy può essere riscritta come:

$$\text{accuracy} = \frac{tp + tn}{tp + tn + fp + fn}.$$

8.5.5 Precision e recall

La **precision** misura quanto siano affidabili le predizioni positive del modello.

$$\text{precision} = \frac{tp}{tp + fp}.$$

Una precision elevata indica che pochi campioni negativi vengono classificati come positivi.

La **recall** (o sensitivity) misura la capacità del modello di individuare tutti i campioni positivi:

$$\text{recall} = \frac{tp}{tp + fn}.$$

Una recall elevata indica che pochi positivi vengono persi dal classificatore.

8.5.6 F-measure

Precision e recall spesso risultano in conflitto: infatti aumentando una si tende a ridurre l'altra. Per combinare entrambe le metriche si utilizza la F-measure:

$$F_\beta = \left(1 + \beta^2\right) \frac{\text{precision} \cdot \text{recall}}{\beta^2 \cdot \text{precision} + \text{recall}}.$$

In particolare $\beta = 1$ produce la F_1-score, ossia la media armonica tra precision e recall. L'interpretazione al variare di β è la seguente:

- $\beta > 1$: maggiore importanza alla recall
- $\beta < 1$: maggiore importanza alla precision

8.6 True/False positive rate

Siano $P_{gt} = tp + fn$ e $N_{gt} = tn + fp$ rispettivamente ground truth positives e negatives (il numero totale di esempi positivi/negativi reali). True positive rate TPR e false positive rate FPS sono definiti rispettivamente come:

$$TPR = \frac{tp}{P_{gt}} = \frac{tp}{tp + fn}, \qquad FPR = \frac{fp}{N_{gt}} = \frac{fp}{tn + fp}.$$

Variando la soglia di classificazione (threshold), cambiano tp, fp, tn, fn e quindi l'intera confusion matrix.

8.6.1 ROC Curve

La receiver operating characteristic (ROC) è una curva che rappresenta il comportamento di un classificatore binario al variare della soglia decisionale. La ROC si ottiene tracciando: TPR contro FPR. In particolare avremo sull'asse x FPR e sull'asse y TPR. La ROC curve consente di valutare il classificatore indipendentemente dalla scelta di una specifica threshold.

Chiamiamo AUC (area under the curve) la misura dell'area sottesa dalla ROC curve. I valori tipici sono $AUC = 1$ che indica un classificatore perfetto e $AUC = .5$ che indica un classificatore casuale. In termini probabilistici l'AUC rappresenta la probabilità che un campione positivo riceva uno score maggiore di un campione negativo.

8.7 Balanced accuracy

Molti problemi reali presentano un forte squilibrio tra le classi. Ad esempio nel rilevamento frodi, diagnosi mediche e anomaly detection. In questi casi abbiamo una prevalenza della classe positiva $< 5\%$. In presenza di class imbalance (classi con squilibrio tra loro) l'accuracy può essere fuorviante. Ad esempio se il 95% dei dati appartiene alla classe

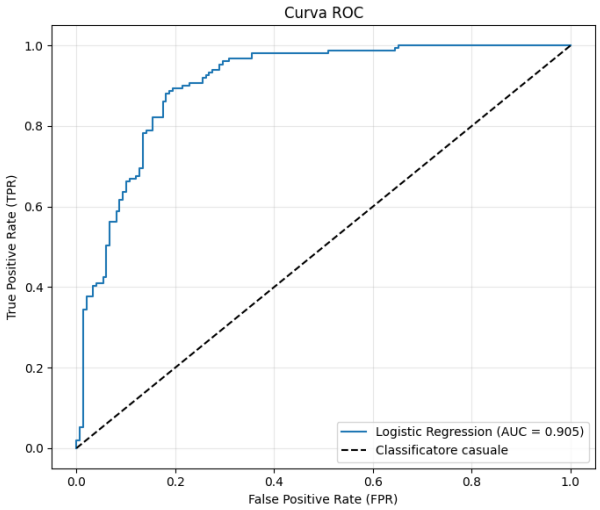


Figura 9: Esempio curva ROC

negativa, un classificatore che predice sempre negativo ottiene il 95% di accuracy pur essendo inutile.

Per affrontare questo problema si introduce la balanced accuracy, che assegna uguale importanza alle due classi. Nel caso binario:

$$\text{balanced-accuracy} = \frac{1}{2}\left(\frac{tp}{tp+fn} + \frac{tn}{tn+fp}\right) = \frac{1}{2}(TPR + TNR),$$

dove TNR è la specificity. Questa metrica evita stime artificialmente elevate ed è più robusta su dataset sbilanciati.